
Image Analysis

An Open-Source Summary

Thomas Debelle



June 10, 2026

Contents

I. Classic Image Analysis	3
1. Introduction	3
1.1. The basics	3
1.1.1. Light	3
1.1.2. Perception	3
1.1.3. Interaction with matter	3
2. Acquisition of images	7
2.1. Image formation - Lens	8
2.2. Sensor array	9
2.2.1. CCD vs CMOS	9
2.2.2. Colour cameras	10
2.3. Perspective Projection - Image Plane	11
2.3.1. Projections limitations	12
2.4. Deviations from the lens model	14
2.4.1. Radial distortion	15
2.4.2. Chromatic aberration	16
2.4.3. Photometric camera model	16
3. Sampling and Quantisation	16
3.1. Sampling	18
3.1.1. Integrate brightness over the cell window	18
3.2. Aliasing	21
4. Enhancement and feature detection	23
4.1. Enhancement - better look of image	23
4.1.1. Histogram Equalisation	24
4.1.2. Deblurring or Unsharp masking	24
4.1.3. Inverse Filtering	25
4.1.4. Noise Suppression	25
4.2. Features - edge and corner detections	25
4.2.1. Binomial filters	26
4.2.2. Median	27
4.2.3. Anisotropic filters - all-in-on of filtering	28
4.3. Enhancement and Feature Detection	28
4.3.1. Edges	28
4.3.2. Gradient	29
4.3.3. Zero-crossing	30
4.3.4. Harris corner detection	31
5. Image decomposition	33
5.1. PCA	33
5.1.1. Method	34

5.1.2. Usage	36
5.1.3. Limitations	37
6. Surface features color and texture	37
6.1. Color	37
6.1.1. Commission Internationale de l'Éclairage (CIE)	38
6.1.2. Constancy	40
6.2. Texture	40
6.2.1. Fourier features	40
6.2.2. Cooccurrence matrices	41
6.2.3. Filter banks	42
7. Motion Extraction	43
7.1. Definition	43
7.1.1. Optical flow	43
7.2. Horn & Schunk	45
7.3. Lucas & Kanade	46
7.4. Active contours	47
7.4.1. Energy minimization	48
8. Local features & Feature matching	49
8.1. Detection	50
8.1.1. Rotation invariant detectors	50
8.1.2. Scale invariant detectors	50
8.1.3. Affine invariant detectors	51
8.2. Description	52
8.2.1. Scale Invariant Feature Transform (SIFT)	52
8.3. Matching	53
9. 3D Acquisition	53
9.1. Passive: no interaction with the scene	53
9.1.1. Uni-directional	53
9.1.2. Multi-directional	54
9.2. Active: interere with the scene to get information	58
9.2.1. Uni-directional	58
9.2.2. Multi-directional	58
II. Modern Image Analysis	60
10. License	70
10.1. Modification	70
10.2. Take down	70

Part I.

Classic Image Analysis

1. Introduction

The key take away is that vision is subjective to the human being and themselves. It is the results of continuous evolution and survival of the fittest. We are more sensitive to light and certain colors, to motion, ... And this fact will be used for efficient image representation and analysis.

We can think of the classic illusion where our brain will interpret certain lengths of an arrow longer than others. Context also does matter, even in a blurry picture our brain will train to make up for what he sees.

1.1. The basics

We first need light to have vision. Light is influenced by object and create perception. ADD image slide 51. The light source bounce on the object and some scattered ray will get trough the 2D Image plane, which will be casted on the sensor array thanks to the lens system.

1.1.1. Light

We will look at it from a physical optic (not geometrical (constant speed and straight path) nor quantum). Light is an Electro-magnetic wave which we assume it to be planar. aka $\frac{|E|}{|H|} = \eta$ in simple term, light is perpendicular. This is a **self-sustaining process** that is characterized by:

1. λ : wavelength
2. Direction
3. E : Amplitude
4. φ : phase
5. Direction of polarisation

Violet: low frequency ($380nm$); Red: high frequency ($760nm$). Remember $\lambda = \frac{c}{f}$.

1.1.2. Perception

We don't perceive light equally. Humans have 3 cones that are not evenly spread apart. But the human vision is trained to compensate for it so everything feels similar in intensity to us. Some animals can have more cones.

1.1.3. Interaction with matter

Table 1: The four types of interaction

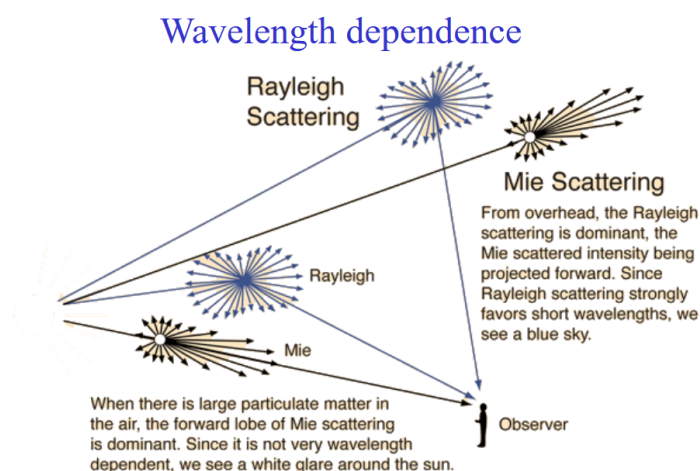
Phenomenon	Example
Absorption	Blue water, green leaf
Scattering	blue sky, red sunset
Reflection	colored in k
Refraction	dispersion by a prism

1.1.3.1. Absorption Dissipation of λ specific for the medium. Resonance in molecules.

1.1.3.2. Scattering When light (radiation/particles) deviates from a straight trajectory by local non-uniformities. Type depends on the relative size of particles and wavelengths:

1. **Rayleigh:** small (λ dependent)
2. **Mie:** comparable (weakly λ dependent)
3. **Non-selective:** large (λ independent)

We ignored transmission and diffraction.

**Figure 1:** Wavelength dependence for scattering

We need scatter to get the bluesky (Rayleigh with Tyndall effect) or it would be dark. Coloured cloud from volcanic eruption = Mie (particle all over the air). Grey cloud is due to non-selective. Less scatter for lower frequency light (infrared).

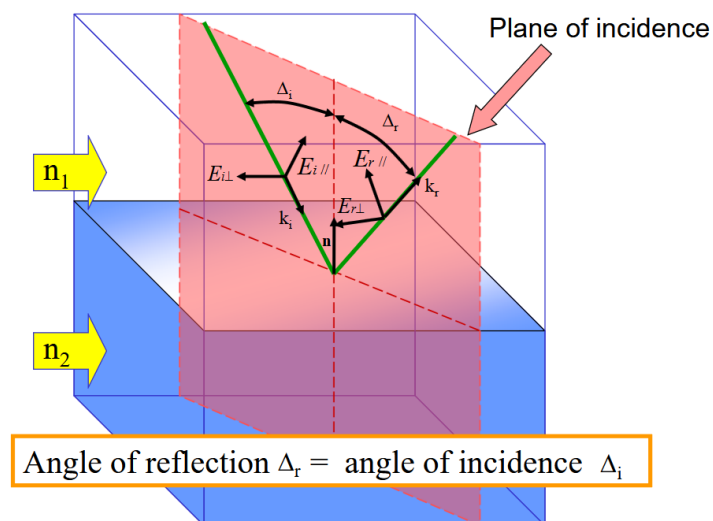


Figure 2: Mirror reflection

1.1.3.3. Reflection With reflection, the notion of polarization is important. The polarization effect creates the fact that a reflected wave on certain surfaces can be reflected with all but one direction of the electric field. This effect is more prominent on **non-metallic** surfaces!

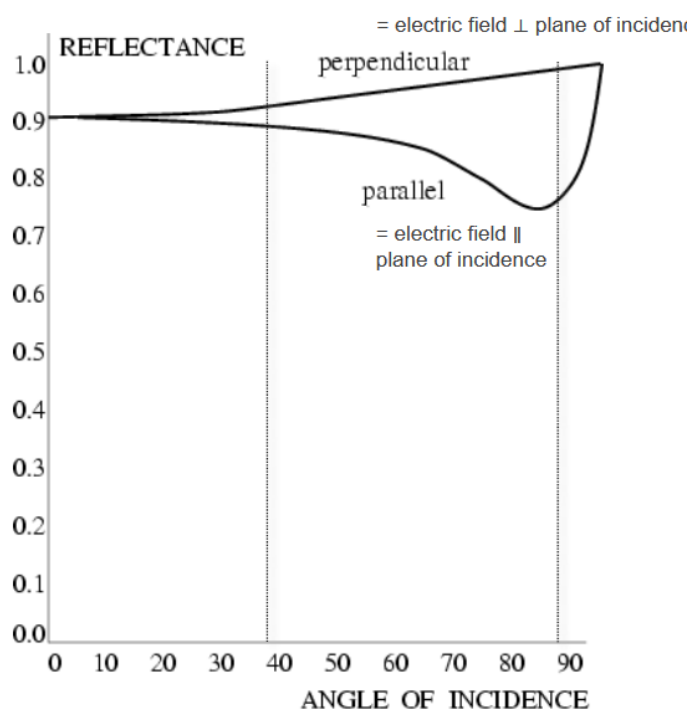


Figure 3: Metallic reflection - strong reflectors

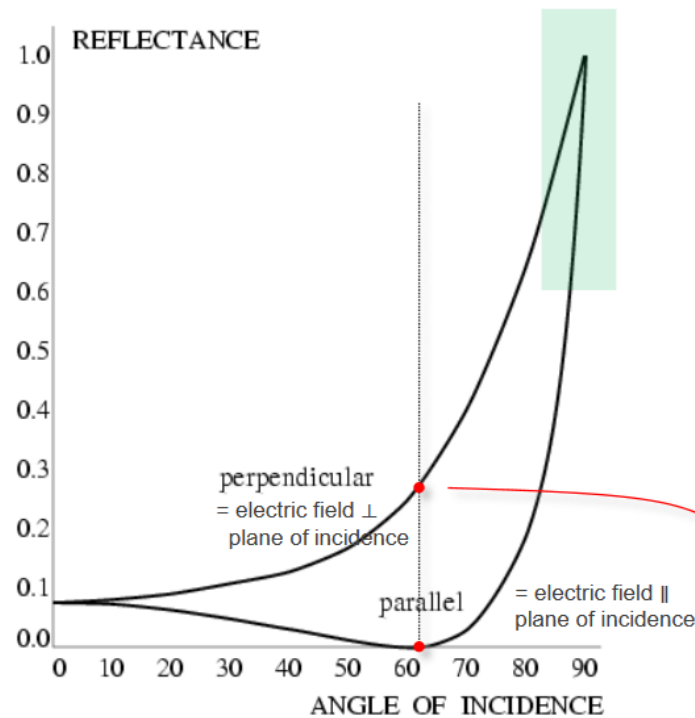


Figure 4: Non-metallic reflection - Red dot: Brewster angle (polarizer) - Blue zone: Grazing angles

Rough surfaces will lead to **diffuse** reflection as the reflection angles will be all over the place. On the other hand, smooth surfaces give **specular** reflection. Both **diffuse** and **specular** can also happen all together.

Reflectance varies through λ and different side of vegetation can typically give various reflection¹

1.1.3.4. Refraction Typically, refraction which is the bending of the light and dispersion which is a frequency dependency.

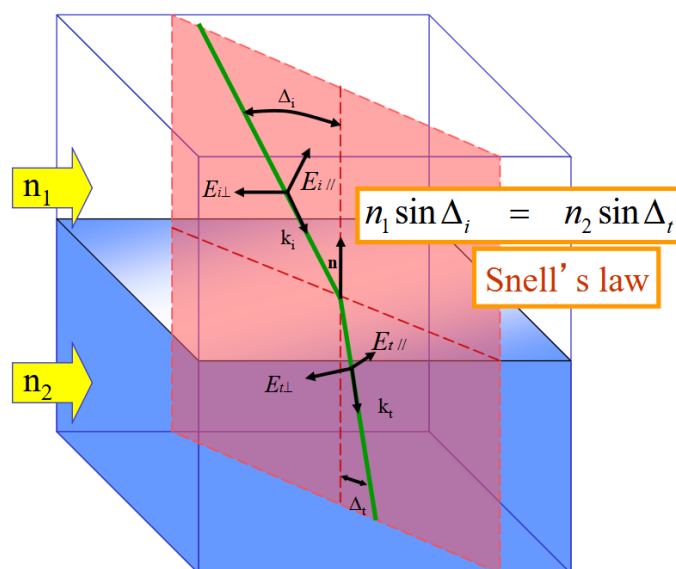


Figure 5: Refraction - governed by Snell's law property of the medium **and** material

¹This fact can be used for spectral reflectance of vegetation since it forms a signature

2. Acquisition of images

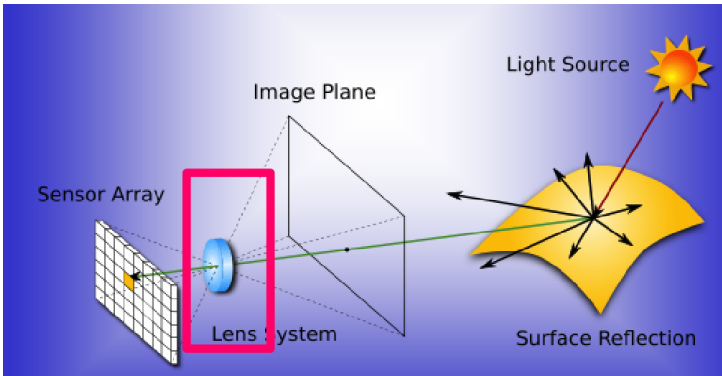


Figure 6: Image acquisition model

Controlling the lights is essential for acquisition as it is the main vector to acquire images. Several techniques exist

Type	Description	Examples
Back-lighting	Light source behind object (better with a diffuser plate)	High contrast silhouette. Binary vision , inspection
Directional	Tilt lights to generate sharp shadow of specular reflection (rough surfaces)	Detection of cracks
Diffuse	Uniform lighting, all directions contribute to the diffusion reflection but specular spike due to spike	Detection of cracks
Polarized: Contrast diffuse and specular	diffuse makes light non-polarized while specular keeps polarized. Put polarizer/analyzer orthogonal, better for glare (high dynamic range) issue.	Detection of cracks
Polarized: Contrast dielectrics and metal	Dielectric at Brewster angle can be used as a polarizer which is not possible for metallic surface (see previous chapter).	So use a polarizer to suppress specular reflection from dielectric OR distinguish between specular and dielectric surfaces
Coloured	Highlight region of similar colour. Use laser (monochromatic light), differentiation between specular and diffuse. Comparing colours. Spectral sensitivity of a sensor.	
Structured	Spatially or temporally modulated light pattern. Projection on a 3D object will distort the projection pattern	3D reconstruction
Stroboscopic	High intensity light flash. Eliminate motion blur.	For high speed inspection

Note about the dark and bright field. Mostly relevant for the Directional/Diffuse lighting. *Dark field*, when the camera is placed outside the specular reflection area of the normal area, only specular reflection (different orientation from normal)

will show up. On the other hand, *Bright field*, we place camera inside this specular reflection light area and the non-normal area will appear dark.

2.1. Image formation - Lens

We use the pinhole model in this class:

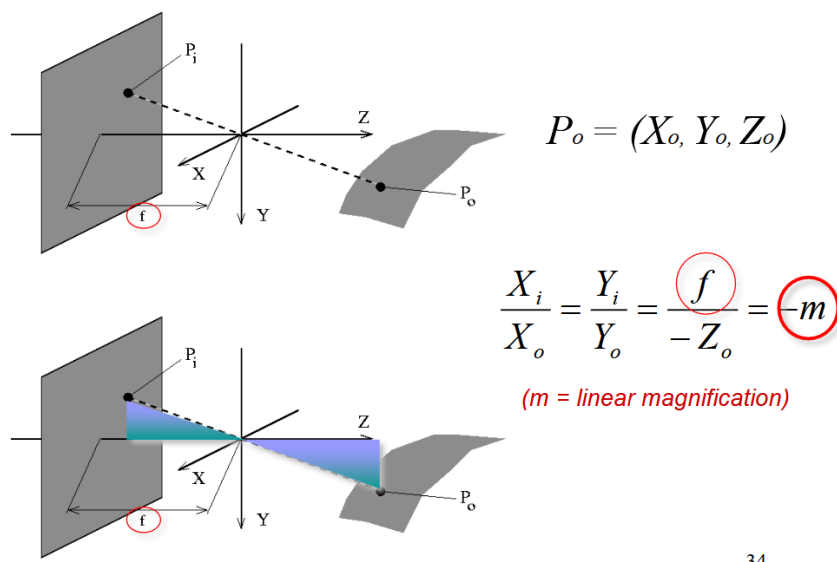


Figure 7: Pinhole model

From this we can use the thin-lens equation which assumes:

- Spherical lens surfaces
- Incoming light $\pm \parallel$ to axis
- Thickness $\ll$ radii
- Same refractive index on both sides

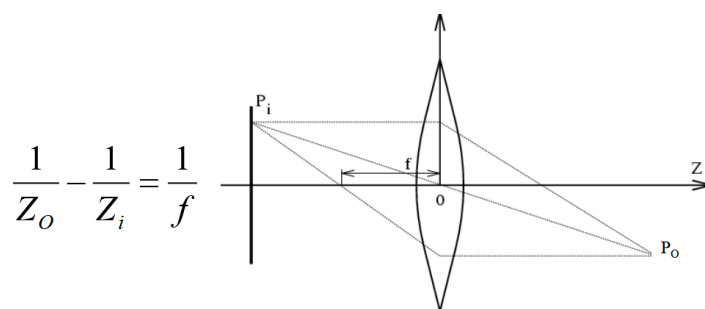
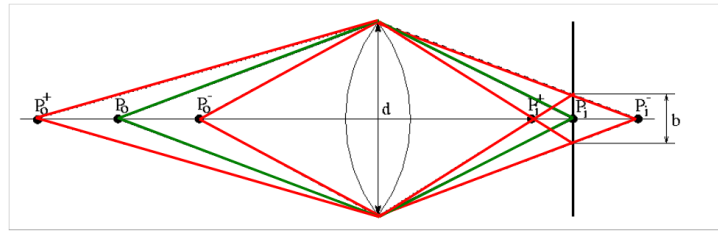


Figure 8: Thin-lens equation



$$\Delta Z_0^- = Z_0 - Z_0^- = \frac{Z_0(Z_0 - f)}{Z_0 + f d/b - f}$$

Figure 9: Depth of field of the thin lens

There is one focal point. Strike a balance between incoming light (d) and large depth-of-field. Smaller depth-of-field is present in microscopes for example.

2.2. Sensor array

2.2.1. CCD vs CMOS

We have 2 types of sensors:

- CCD: Charge-coupled device
 - Old school
 - Charges must hop from tile to tile
 - Cover the total pixel (high fill rate)
 - Expensive
 - Blooming (due to the charge hoping)
 - High power consumption
- CMOS: integrated
 - Cheap, Can put other component on the chip (integration)
 - Read line by line (like RAM = random access)
 - Smaller
 - Low power
 - Same sensor as CCD but each has their own amplifier = more noise + not covering the full space = lower fill rate / sensitivity

They both works in 2 steps:

1. Photon to electron (charge) conversion
2. Electron (charge) to voltage conversion

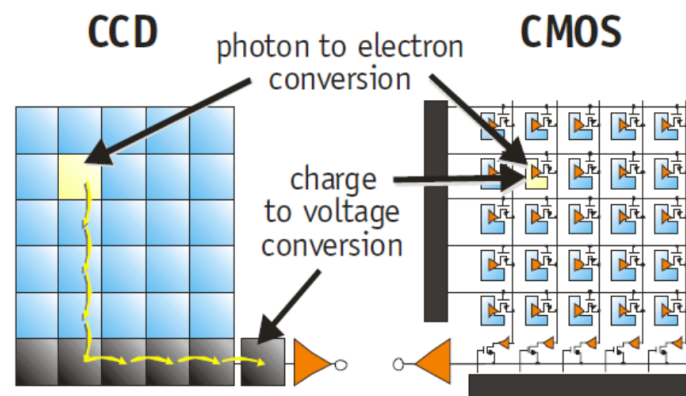


Figure 10: CCD vs CMOS

2.2.2. Colour cameras

2.2.2.1. 1. Prism The idea is to split the light using *dichroic prism* to split it in 3 colors. This requires really good precision but allows for good color separation.

2.2.2.2. 2. Filter mosaic Back to the *bayer pattern* introduced earlier. Good for tricking the human eyes. Each pixel will have a filter in front to follow the bayer pattern. This is good but can create some aliasing especially when an edge is right on a pixel array causing some extra inexistant colors (aliasing).

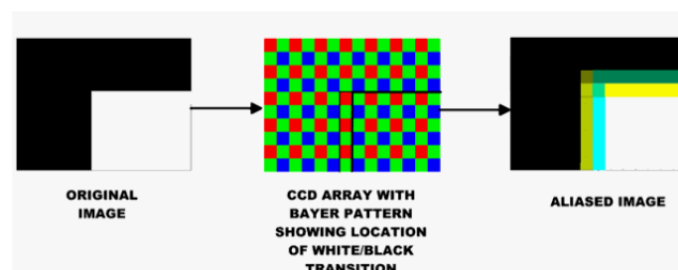


Figure 11: Aliasing with bayer pattern

We loose actual resolution since now we need 2x2 pixel array to get 1 colored pixel ! We have to stack even more density but need to capture more lights → Use microlenses.

2.2.2.3. 3. Filter wheel Here instead of having 3 filters like in a filter mosaic, we have a full wheel with many filters. Behind the wheel, there is a static sensor (**only static scenes**).

2.2.2.4. Summary

Table 3: Summary of the 3 coloured camera

Type	Prism	Mosaic	Wheel
# Sensors	3	1	1

Type	Prism	Mosaic	Wheel
Resolution	High	Medium	Good
Cost	High	Low	Medium
Framerate	High	High	Low
Artefacts	Low	Aliasing	Aliasing (just motion of wheel)
Bands (color)	3	3	3 or +
Application	High-end cameras	Low-end cameras	Scientific applications

2.2.2.5. Improvements One issue is the Aliasing problem for object in motion. Since we read line by line in CMOS camera, for fast turning object (turning at $\geq f_s/2$) we have aliasing. To solve this, we need a global shutter instead of reading line by line.

2.3. Perspective Projection - Image Plane

Here, we revisit the pinhole model where now, we first project everything on an object plane and then this will get through the lens. The center of the lens is the center of the projection, we have an virtual image plane which is also called **perspective projection** (we flattened out everything from the outside on it):

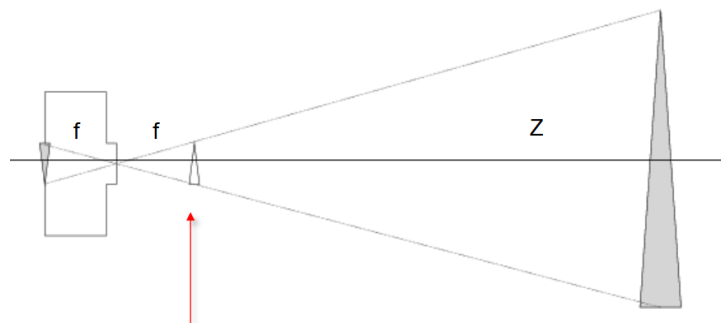


Figure 12: Camera projection model

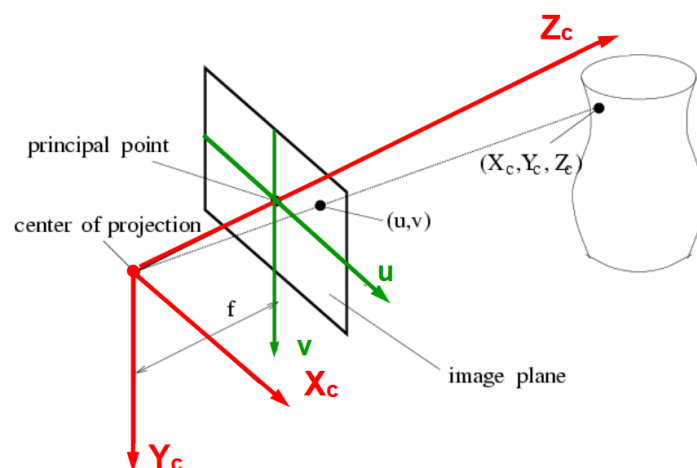


Figure 13: 3D view of the camera projection model

The origin is the center of the projection (X_c row-pixel, Y_c column-pixel space), the Z_c axis correspond with the optical axis.

If we look at the u, v space we have the following relationship, which comes from the “similar triangles property” where you look once from above (for X case) and one from the side (for Y case).

$$u = f \frac{X_c}{Z_c} \quad v = f \frac{Y_c}{Z_c}$$

We can then say that if Z is constant, we can write $x = kX_c$ and $y = kY_c$ where $k = f/Z$. Orthographic projection + a scaling. This forms a good approximation if objects are small compared to their distance from the camera.

2.3.1. Projections limitations



Figure 14: Pseudo-orthographic vs Perspective projection

The perspective projection model is incomplete. What about world coordinate ? What if we want our u, v coordinates as a row and column numbers (like in a matrix of pixels).

This part of the class that does not form exam material but helps with understanding where does those matrices come from.

2.3.1.1. The (u, v) projection to pixel We can see a matrix of pixel instead of the (u, v) projection. We just need 3 informations: 1. Center of matrix x_0 and y_0 , 2. Width of array k_x and 3. Height of array k_y giving us:

$$\begin{cases} x &= k_x u + x_0 + sv \\ y &= k_y v + y_0 \end{cases} \quad (1)$$

Note: the s is for skew, we have the aspect ratio k_y/k_x^2

2.3.1.2. The global world coordinate problem Those equations are based on the previous $u = fX/Z$ and $v = fY/Z$. Here $\langle, \rangle$ represents the dot product $\cdot$ between 2 vectors.

²I thought it was the opposite...

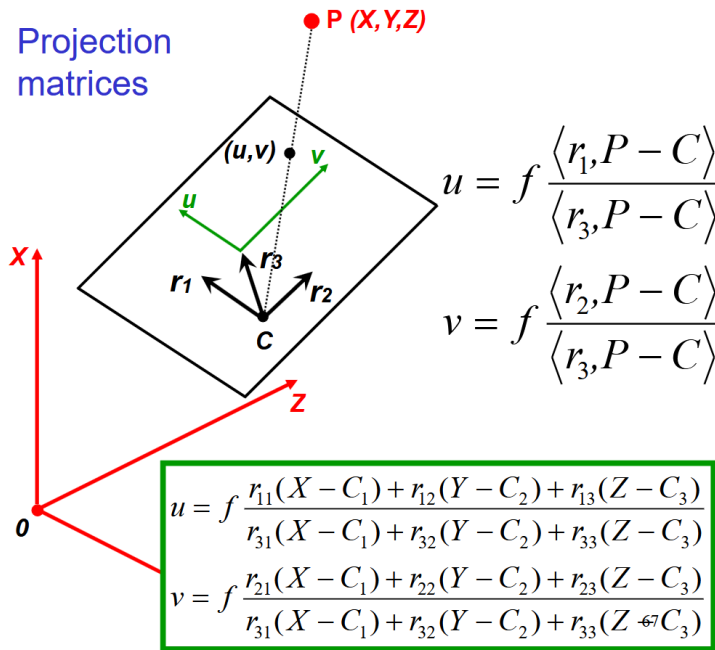


Figure 15: Projection matrices

The $P - c$ represents the dotted vector and C being the camera position in a global coordinate system (the camera is no longer stuck to the 0,0 point). We then use the dot product between one of its axes and the vector $P - C$ to obtain the value in this specific vector dimension (a bit like the X before). Repeat it for Y and Z .

2.3.1.3. Homogeneous coordinates Capturing a 2D image requires to navigate a 3D world (and so on for 3D and 4D). This is important in computer vision where we have to deal with projection where we **have to** divide a point by z !

$$\tau \begin{pmatrix} x \\ y \\ z \end{pmatrix} \rightarrow_{\text{capturing/projection}} \begin{pmatrix} x/z \\ y/z \end{pmatrix}$$

2.3.1.4. Putting it all together We can rewrite the previous big dot/inner product using the homogeneous coordinate (removes division) as:

$$\tau \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} fr_{11} & fr_{12} & fr_{13} \\ fr_{21} & fr_{22} & fr_{23} \\ r_{31} & r_{12} & r_{13} \end{pmatrix} \begin{pmatrix} X - C_1 \\ Y - C_2 \\ Z - C_3 \end{pmatrix}$$

we can then apply the pixel coordinate system on this equation:

$$\tau \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} k_x & s & x_0 \\ 0 & k_y & y_0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} fr_{11} & fr_{12} & fr_{13} \\ fr_{21} & fr_{22} & fr_{23} \\ r_{31} & r_{12} & r_{13} \end{pmatrix} \begin{pmatrix} X - C_1 \\ Y - C_2 \\ Z - C_3 \end{pmatrix}$$

Back to important material for exam.

we can further simplify with

$$\tau \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \overbrace{\begin{pmatrix} k_x & s & x_0 \\ 0 & k_y & y_0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{pmatrix}}^{K \text{ Calibration matrix}} \overbrace{\begin{pmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{pmatrix}}^{R \text{ Camera rotation}} \begin{pmatrix} X - C_1 \\ Y - C_2 \\ Z - C_3 \end{pmatrix}$$

We then define the following vectors:

$$p = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad P = \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \quad \tilde{P} = \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix} \quad (2)$$

Which then yields the simplified version:

$$\rho p = K R^\top (P - C) \quad \text{some non-zero } \rho \in \mathbb{R}$$

$$\text{or, } \rho p = K(R^\top | - R^\top C) \tilde{P}$$

$$\text{or, or, } \rho p = (M|t) \tilde{P} \quad \text{with rank } M = 3$$

2.4. Deviations from the lens model

We assume:

1. All rays from a point are focused onto 1 image point
2. All image points in a single plane
3. Magnification is constant

If those are not respected, we have **aberrations**! Aberrations can be **geometrical** (small for paraxial rays) or **chromatic** (refractive index function of λ - Snell's law)

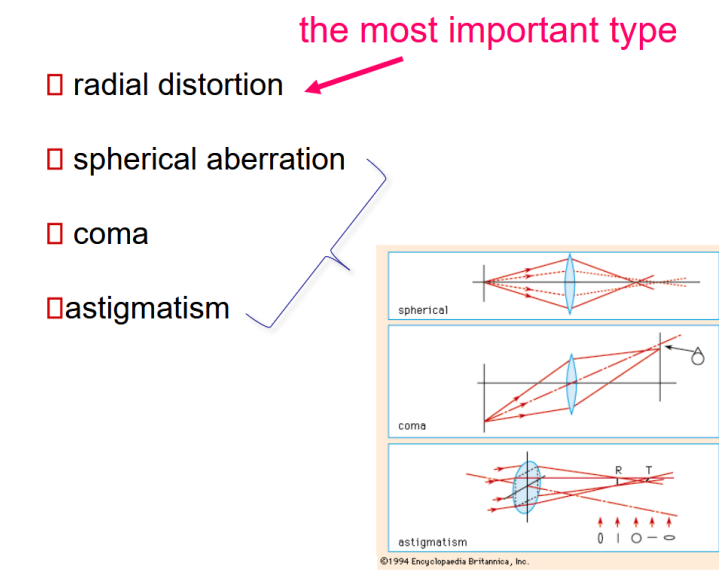


Figure 16: Geometrical aberrations

They all cause issues on the focus point of the image plane. Spherical appears when the bending of light is not uniform throughout the lens, typically outer ray converges sooner (small focal length) than inner one. They do not converge all together. Coma is a bit like spherical but this time with a tilted ray. Astigmatism is problems between the *tangential* and *sagittal* (90° rotation from tangential) focus point which are not equal.

2.4.1. Radial distortion

By far the most important type of distortion which causes “fish-eye” look or pincushion. It creates various *magnification* for different angles of inclination. The pixel either sinks towards the center or slides outside along the line connection the center and the original pixel.

magnification different for different angles of inclination



Figure 17: Radial Distortion

We can model this sliding distance d with the following equation and thus can be corrected. Some algorithm looks at human made structure (straight lines) to establish the effect.

$$d = (1 + \kappa_1 r^2 + \kappa_2 r^4 + \dots)$$

2.4.2. Chromatic aberration

Rays of different λ will focus in different planes. This creates various colours especially along sharp edges. This **can't** be removed completely but *achromatization* can be achieved by selecting the right type of glasses.

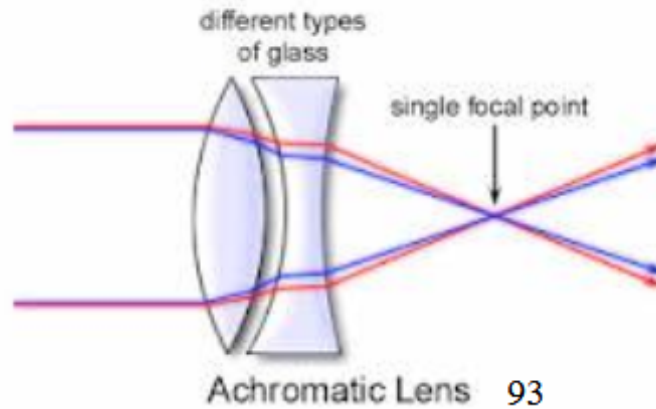


Figure 18: Achromatization

2.4.3. Photometric camera model

We want to convert the object radiance into a pixel grey level. This is a 2-step process with our idea of the image plane. We first project the radiance of the object to the image, then this image irradiance projects to a pixel in a grey level. The **cos⁴** law governs the radiance. The more an object is off-axis from the center by an angle θ the more it reduces its irradiance in the pixel. This materializes with **natural vignetting**³ (like the old instagram filter).

$$I = R \frac{A_l}{f^2} \cos^4 \theta$$

This will get converted in a grey pixel with:

$$pix = gI^\gamma + d$$

With the g of the camera, d the dark reference of the camera. The gamma γ is a non-linear relationship which changes the mid-tone without really changing the pure black or white. A value lower than 1 makes the image lighter and vice-versa. Nowadays, cameras have become quite linear meaning that $\gamma = 1$.

3. Sampling and Quantisation

We live in an analog world but our computers and digital world is finite. So we have to sample (Discretize points) and quantize (attribute them a finite value).

From what we have seen and learned before, we can trick the human eye to see a higher quality image than it is actually. For example the display on a phone has a RGB pixel ratio as 1:2:1 because the human is less sensible to green. We call

³optical vignetting appears due to an obstruction of the lens by an element

this grid the **bayer filter**. Other representation exist which can be more exotic, e.g.: hexagonal (more isotropic, similar structure as in the retina, no connectivity ambiguities).

We have quantization where we usually talk about levels. Typically a n bits representation has 2^n levels. We can also apply fancier quantization scheme instead of simple linear one. The most classic representation is the RGB888 (1 byte per pixel for each color).

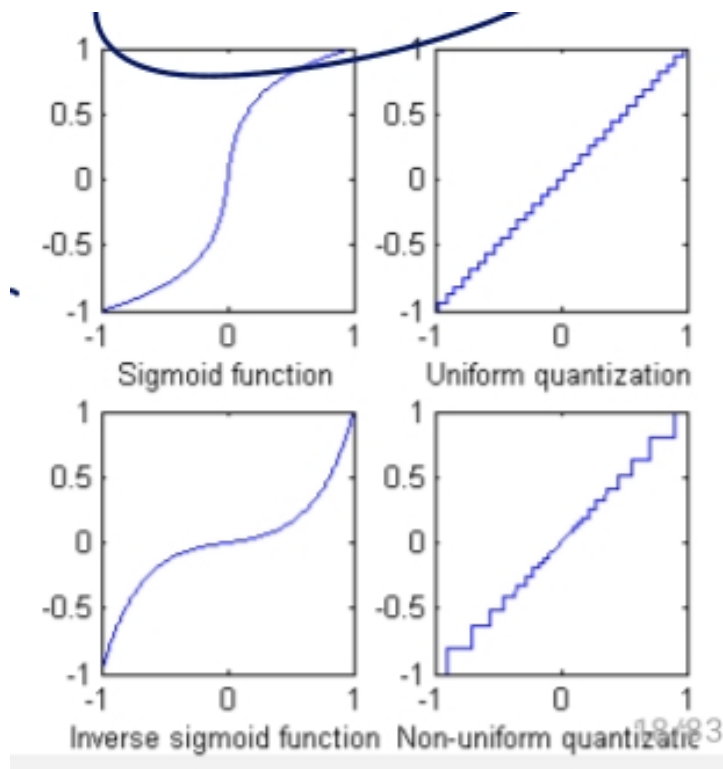
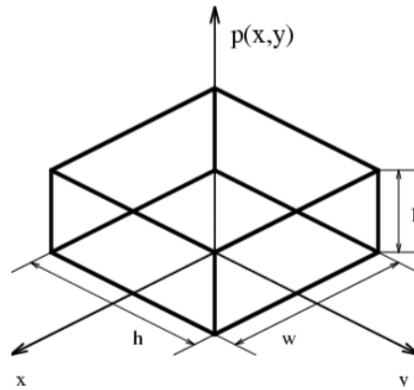


Figure 19: Various quantization scheme

3.0.0.1. HDR High Dynamic Range (HDR) is a technology where we have various luminance on the screen giving really good contrast in dark and bright area. Usually, we combine multiple photos with various gains and then recombine it all together with the various exposure.

3.1. Sampling

3.1.1. Integrate brightness over the cell window



$$o(x', y') = \iint i(x, y) p(x - x', y - y') dx dy$$

This is **convolution**: $i(x, y) * p(-x, -y)$

Figure 20: Integrating over a pixel cell

This forms a convolution. Convolution are **linear**, $f_1 \rightarrow g_1 \quad f_2 \rightarrow g_2 \Rightarrow af_1 + bf_2 \rightarrow ag_1 + bg_2$, and **shift-invariant**, $f(x, y) \rightarrow g(x, y) \Rightarrow f(x - a, y - b) \rightarrow g(x - a, y - b)$.

Convolutions are also commutative and associative as proven in the Fourier domain.

$$\mathfrak{F}(u) = \int_{-\infty}^{\infty} f(t) e^{-2i\pi xu} dx$$

For the 2D case, we have $e^{2i\pi(ux+vy)}$ which has for Euler form $\cos(2\pi(ux + vy)) + i \sin(2\pi(ux + vy))$. Now, the u and v represents the frequency along the x and y dimensions. To obtain the λ of the signal we need to take the norm of the frequency for both axis inverted:

$$\lambda = \frac{1}{\sqrt{u^2 + v^2}}$$

As in 1D, we can do Fourier transform to decomposition the image in a bunch of frequencies. As in 1D, we have a complex results where the magnitude is $|\mathfrak{F}(u, v)| = \sqrt{\mathbb{R}(\mathfrak{F}(u, v))^2 + \mathbb{I}(\mathfrak{F}(u, v))^2}$ and the phase angle $\angle \mathfrak{F}(u, v) = \arctan(\mathbb{I}(\mathfrak{F}(u, v))/\mathbb{R}(\mathfrak{F}(u, v)))$.

$$\mathfrak{F}(u, v) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) e^{-2i\pi(xu+yv)} dx dy$$

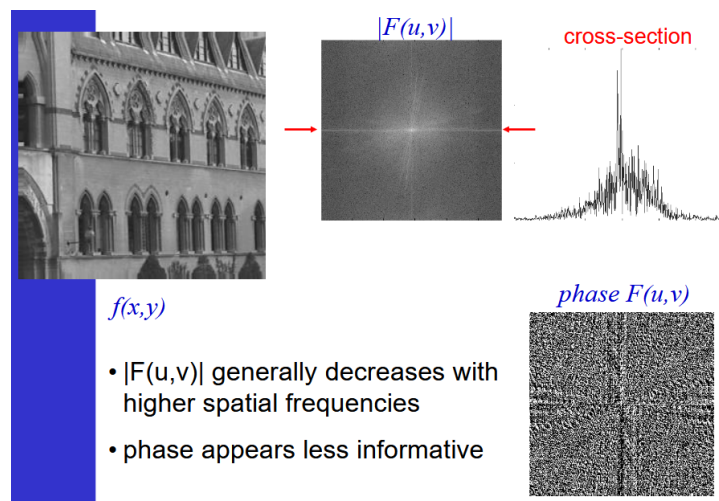


Figure 21: Fourier decomposition of images

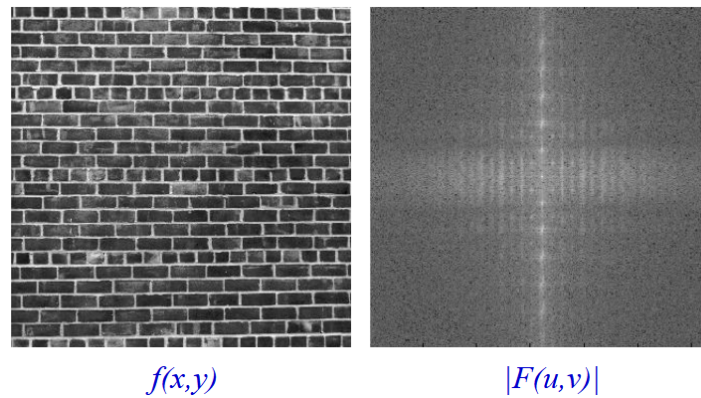


Figure 22: Repetitive patterns in an image creates repetitions in frequency domain

Removing patterns peak in frequency allows to remove periodic background.

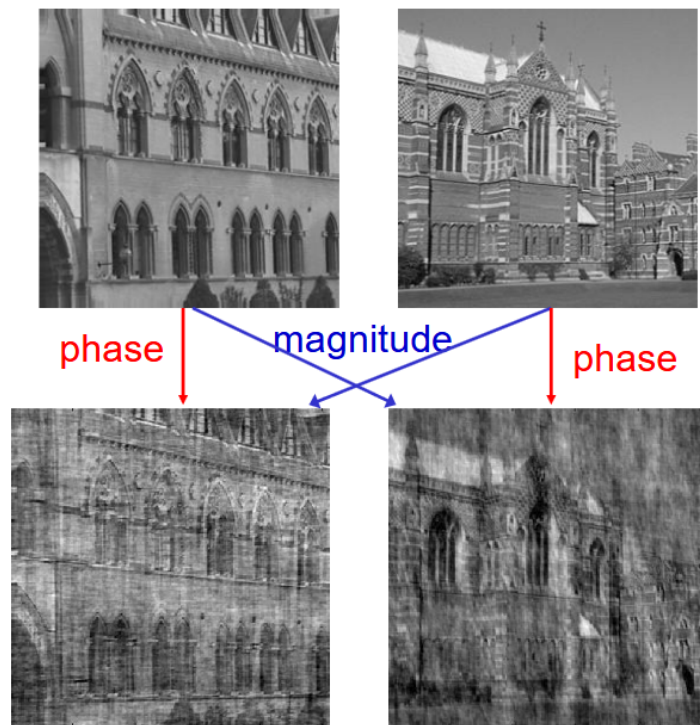


Figure 23: Mixmatch of phase and magnitude

3.1.1.1. Properties of spatial-frequency domain

Spatial domain	Frequency domain
Real	Real part → even; Imaginary part → odd
Real, even	Real, even (cosine)
Real, odd	Imaginary, odd (sine)

Important to remember that a convolution in spatial domain is a multiplication in the frequency domain and vice versa.

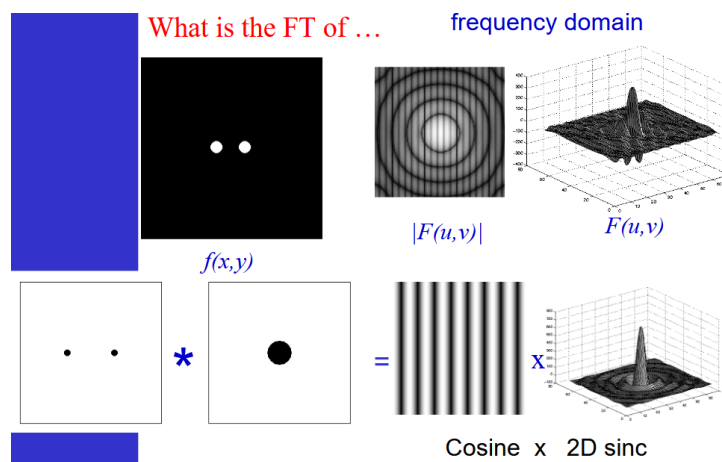


Figure 24: FFT approach

We have the base image (the dot) and the the transfer function, MTF R, which contains the amplification and phase shift information to every frequency in the input image I.

3.1.1.2. Integrating with the convolution Re-using the integration over a cell window:

$$o(x', y') = \int \int i(x, y) p(x - x', y - y') dx dy \leftrightarrow i(x, y) * p(-x, -y)$$

We can then transform this $p(x, y)$ round-point (as seen in the previous figure) in a $P(u, v)$

$$P(u, v) = \int_{-\infty}^{\infty} e^{-i2\pi(ux+vy)} p(x, y) dx dy \quad (3)$$

$$= wh \left(\frac{\sin(\pi w u)}{\pi w u} \right) \underbrace{\left(\frac{\sin(\pi h v)}{\pi h v} \right)}_{\text{sinc}} \quad (4)$$

The sinc function is predominantly low-pass **and** non-causal (no phase shift) and it has phase reversal.

3.2. Aliasing

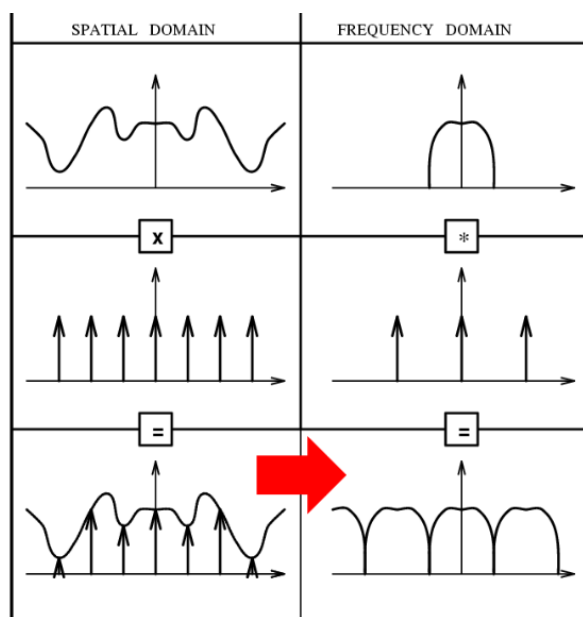
This corresponds to a 2D Dirac train convolution on the grid like:

$$f(a, b) = \int \int f(x, y) \delta(x - a, y - b) dx dy$$

The multiplication with the 2D pulse train with a spacing of distance w and h:

$$\sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} \delta(x - kw, y - lh) \quad (5)$$

$$\longleftrightarrow \frac{1}{wh} \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} \delta\left(x - k\frac{1}{w}, y - l\frac{1}{h}\right) \quad (6)$$



Sampling in one domain leads to a periodic repetition in the other, here space $\rightarrow$ frequency

Figure 25: Basic sampling theorem

Also, we need to do a Discrete Fourier Transform, DFT, to discretize the space and frequency. We can also use FFT to bring the complexity of the algorithm from N^2 to $N \log_2 N$.

$$\sum_{m=0}^{M-1} \sum_{n=0}^{N-1} F(k, l) e^{2\pi i \left(\frac{km}{M} + \frac{ln}{N} \right)} \quad (7)$$

$$\longleftrightarrow F(k, l) = \frac{1}{MN} \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n) e^{-2\pi i \left(\frac{km}{M} + \frac{ln}{N} \right)} \quad (8)$$

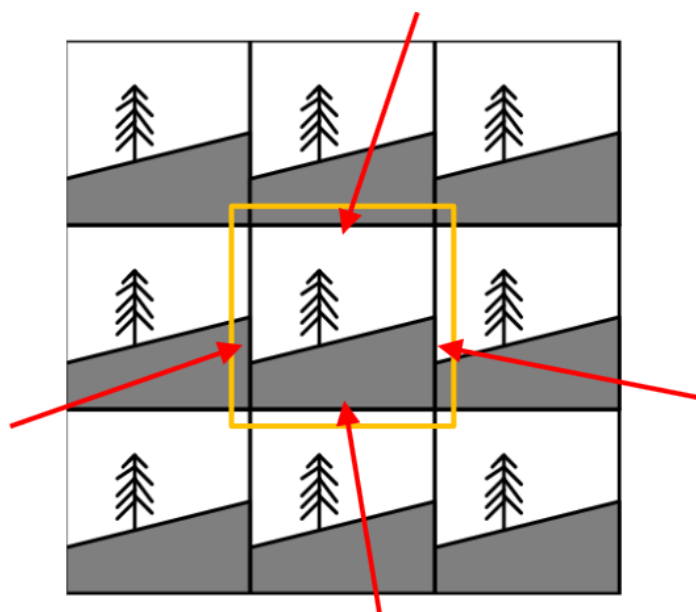


Figure 26: Periodicity assumed in both domains, might introduce false high frequencies at image boundaries

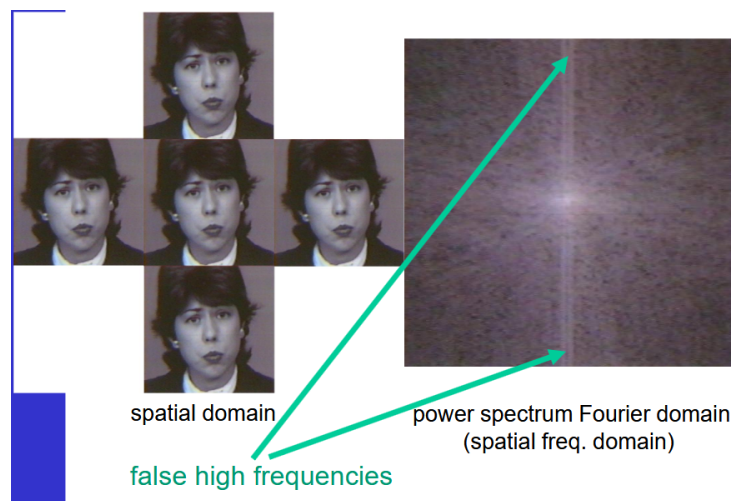


Figure 27: We can have false high frequencies due to sampling

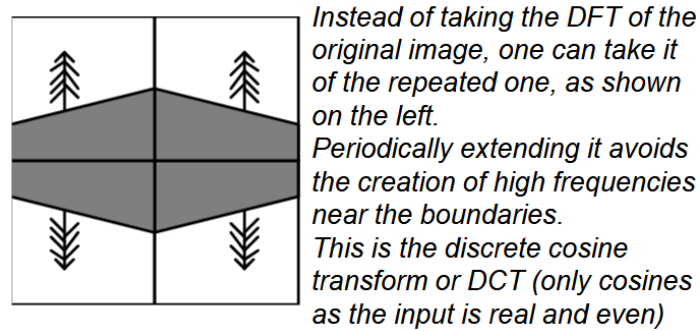


Figure 28: Solution on periodicity

Again, the Shannon theorem is important in 2D as well such that the sampling distance w, h along the x, y axis must follow:

$$w \leq \frac{1}{2w_b} \quad h \leq \frac{1}{2v_b}$$

4. Enhancement and feature detection

Strong from the knowledge of the previous section, we can do some spectral filtering. For example, if we have some white noise in an image we could apply a low pass filter on it to already remove a big part of the noise while losing a marginal part of the actual picture information. Usually, this will sort of blur the image because sharp edges are like high-frequency information.

4.1. Enhancement - better look of image

Typically, HDR is a form of enhancement for better contrast. A cheaper way of doing HDR is doing histogram equalisation which allows for better usage of the full dynamic range available (a bit like gamma correction I think).

4.1.1. Histogram Equalisation

This will makes the image more vibrant if we exploit more of the dynamic range, we **redistribute** the intensities through a **mapping that keeps their relative order**. Ideally, we make the histogram **flat**.

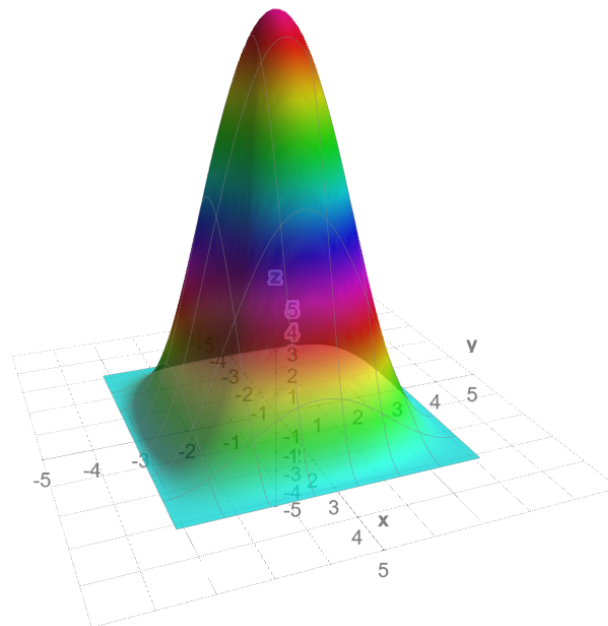


Figure 29: We tune the Cumulative Intensity Probability - or CPD to a flat line

We will compress the flat parts of the CPD, and expand the steepest one. But the main issue, since we move bins, we want have a flat histogram, we will just spread the bins more equally due to the **discrete nature** of the input. On top, neighboring pixels won't always have the same difference of intensities after the remapping (less problematic though).

4.1.2. Deblurring or Unsharp masking

If we deblur we actually increase the image sharpness. It's *simple, linear, image independent and effective*.

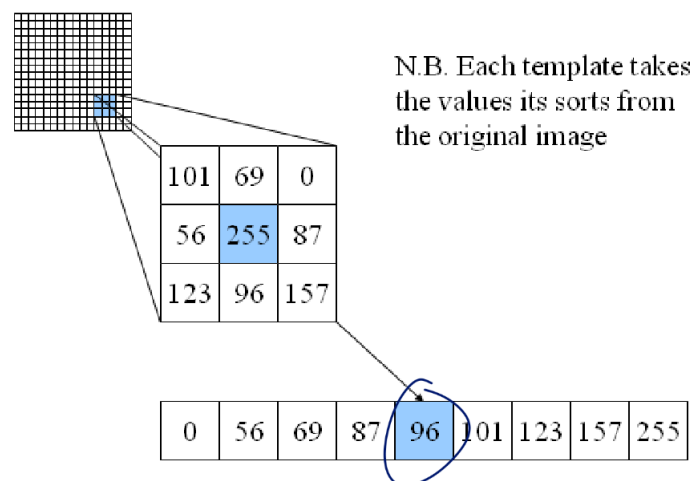


Figure 30: Imagine a vertical cut of image where x-axis is the pixel number and y-axis the grey-level

We see that we, indeed, make the edge steeper. Moreover, the difference between the original-smooth makes a good fit for second order derivative. This is a bit like running a diffusion process backwards.

The little over and under-shoots at the edge even magnifies the contrast (Mach band effect).

4.1.3. Inverse Filtering

Now, we shift gears and go into the frequency. We have the frequency function that blurs the image with $B(u, v)$. To undo it we apply $B'(u, v)$ such that $B'(u, v)B(u, v) = 1$. Two danger here:

1. What happens for $B(u, v) = 0$
 - Use a constant at the denominator $\rightarrow$ Spurious high-frequencies :'
 - $B'(u, v) = B(u, v)/(B(u, v)^2 + C) \rightarrow$ Much better ! Wiener-like reduces C for larger SNR and vice versa.
2. For low $B(u, v)$ after noise was applied $\rightarrow$ may over-fit the noise...

4.1.4. Noise Suppression

4.1.4.1. Low pass As introduced, we can use a low-pass filter for noise suppression. The main problem with applying this rectangular (actually a circle) window is the rippling due to the sinc function as seen here

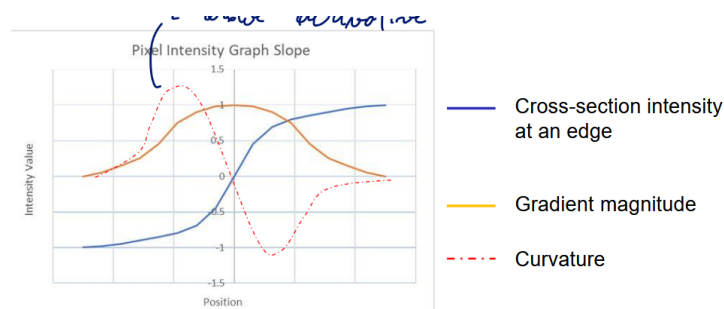


Figure 31: Rippling effect

4.2. Features - edge and corner detections

ABOVE SHOULD BE THE START OF CHAPTER ABOUT ENHANCEMENT AND EDGES

P.41

4.2.0.1. Convolution filters It is based on the insight that noise affects high frequencies the most due to their low SNR. To build a low-pass filter, we will use an averaging kernel such:

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & 1 & 1 \end{pmatrix} * \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}}_{\text{separable filter}}$$

The two $(1, 1, 1)$ vectors can be seen in the *frequency* domain as $1 + 2 \cos(2\pi u)$ for the horizontal domain or $1 + 2 \cos(2\pi v)$ in the vertical one. The intuition for this is to look at the vector as a 1D image centered around the middle value. Finally, the convolution can be simplified in the frequency domain as:

$$(1 + 2 \cos(2\pi u))(1 + 2 \cos(2\pi v))$$

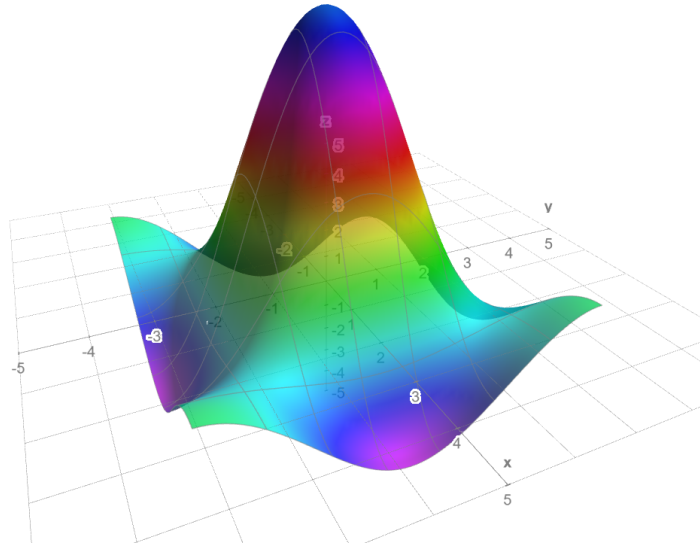


Figure 32: Representation in 3D of the function

The function is purely real and low-pass with some phase reversal! We can again derivate something similar for a 5x5 kernel:

$$(1 + 2 \cos(2\pi u) + 2 \cos(4\pi u))(1 + 2 \cos(2\pi v) + 2 \cos(4\pi v))$$

With this one, the phase reversal is even more significant, the higher the frequency the more it looks like a sinc function. This creates more ripples and disturbing effects. Hard to get no ripple and a real low-pass for higher order functions.

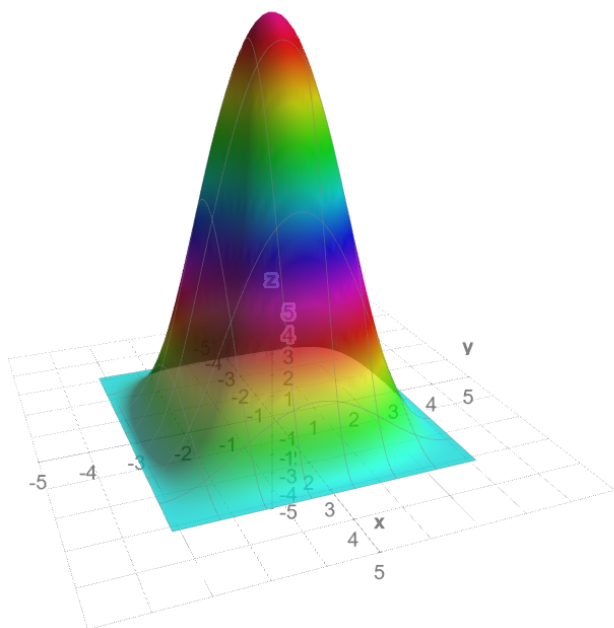
From all those experiments we can draw 3 conclusions:

1. Separable filters can be implement more efficiently
2. Better to go in the frequency domain for large kernel
3. Keep it simple, mask of boolean or power of 2 are cheap!

4.2.1. Binomial filters

They are here to solve the ripple and low-pass issue of the convolution filters presented before. This guarantees no phase reversal.

$$\begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 1 \end{pmatrix} * \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} \longleftrightarrow (2 + 2 \cos(2\pi u))(2 + 2 \cos(2\pi v))$$



{width=50%}

Note that noise removal linear filters will exhibit blurring due the fact they cannot difference noise and actual information.

4.2.2. Median

Instead of averaging over using a kernel of ones, we take the median - the middle point of neighborhood of pixel. We don't create a new grey level, we take the one in the middle.

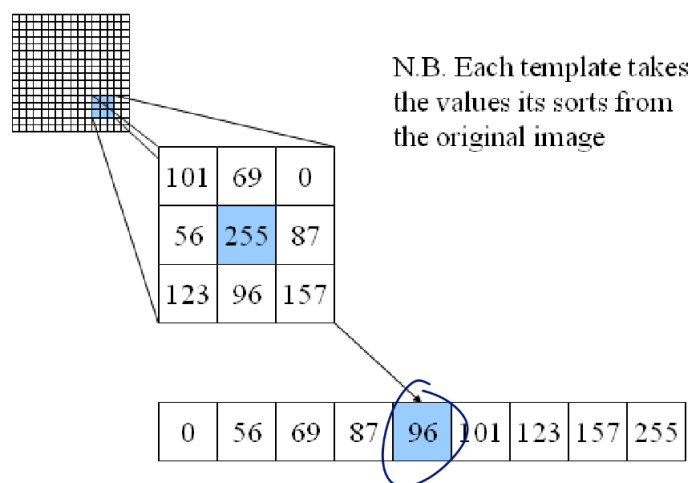


Figure 33: Median filters principle - credits: southampton.ac.uk

One advantage of a median filter is the fact it is robust against outliers and it will preserve the contrast instead of smoothing out all the values and making the image more blurry.

- Discard outliers
- Preserve discontinuities

But, it will loose in details (especially small areas). Moreover, give a patchy effect instead of a smooth uniform color.

4.2.3. Anisotropic filters - all-in-on of filtering

This filter does:

1. Binomial smoothing between edges
2. Unsharp masking at edges

For this, we need to cover edge detection

Table 5: Summary table of the enhancement procedures

Operation name	Description	Linear ?
Histogram equalisation	Re-sort histogram to create a more linear Cumulative Probability Density	V
Unsharp mask		V
Inverse Filtering	Realize the opposite operation. Need to carefully handle low SNR with a constant.	V
Low pass	Removes noise since it affects more high-frequencies due to low SNR	V
Binomial	Like low-pass but avoids phase reversal	V
Median	Take the middle point instead of the mean	X
Anisotropic	Binomial between edges - Unsharp at edge	X

4.3. Enhancement and Feature Detection

4.3.1. Edges

Definition:

Edge correspond to image locations with strong intensity changes due to:

1. Reflectance
2. Orientation of shape normals (its surface) - the object's edge
3. Illuminations (shadow)

The hardest challenge in edge detection is linking up all the pixels into a good line drawing.

The methods cover here will be based on the gradient which gives the direction of the steepest change along as its magnitude being the rate of change:

$$\text{gradient} = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right) \quad \text{magnitude} = \sqrt{\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2}$$

2 methods will be covered using **isotropic** operators⁴:

1. Locating high intensity gradient magnitude

⁴there exists other (and better) operators but too complex for this class

2. Locating inflection points in the intensity profiles

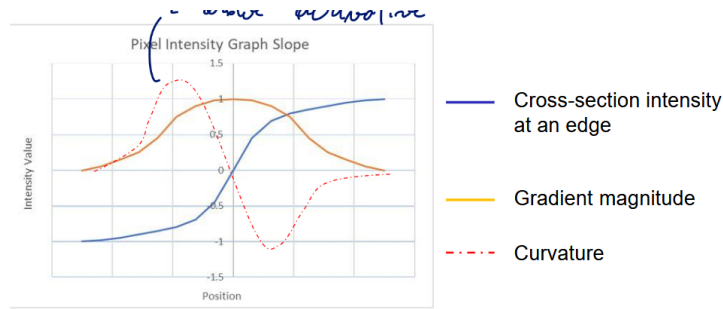


Figure 34: With curvature being a sort of second order derivative

4.3.2. Gradient

The idea is to:

1. measure the gradient **magnitude** of all the steep slopes in the image. This operator is isotropic since there are equal contribution of the x and y direction.
2. Measure the angle θ that maximizes $\frac{\partial f}{\partial x'} = \frac{\partial f}{\partial x} \cos(\theta) + \frac{\partial f}{\partial y} \sin(\theta)$ this yields $\theta_{xtr} = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$

The gradient is a non-linear operator of $\partial/\partial x$ which are linear, shift-invariant ! So this can be implemented using a convolution, the **Sobel masks** which are separable and cheap to implement. The Sobel masks are a bit like a binomial filter but with a derivative in the middle. The idea is to have a strong response for high frequencies (edges) and low response for low frequencies (flat areas):

$$\frac{\partial}{\partial x} = (-1, 0, 1) * (1, 2, 1)^T = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix} \quad \frac{\partial}{\partial y} = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$

From their outputs, we take the square root of the sum of their squares, take the arctan which will give us the orientation of the edge!

Again we can move to the frequency domain where we remember we can see the vector as a 1D image centered around the middle value. The Fourier transform of the Sobel mask is:

$$(-1, 0, 1) \longleftrightarrow 2i \sin(2\pi u) \quad (1, 2, 1) \longleftrightarrow 2 + 2 \cos(2\pi u)$$

We see that it is pass band along the u-dir (red on the graph) and low-pass along the v-dir (green on the graph).

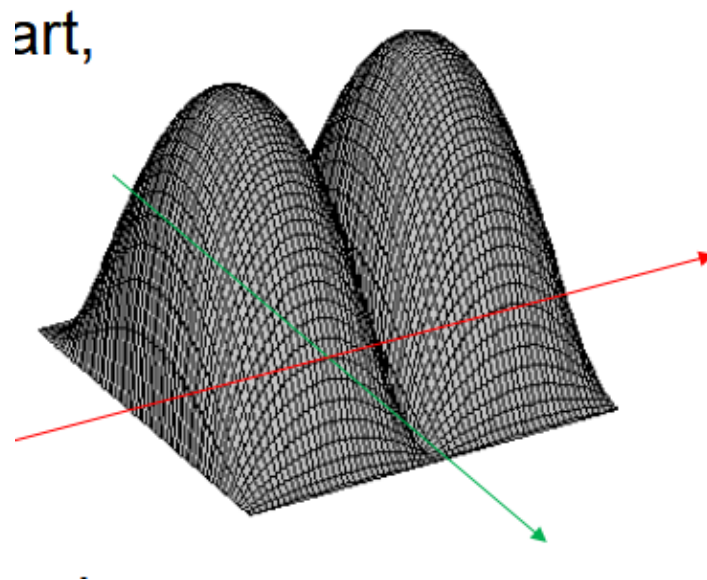


Figure 35: Function in 3D

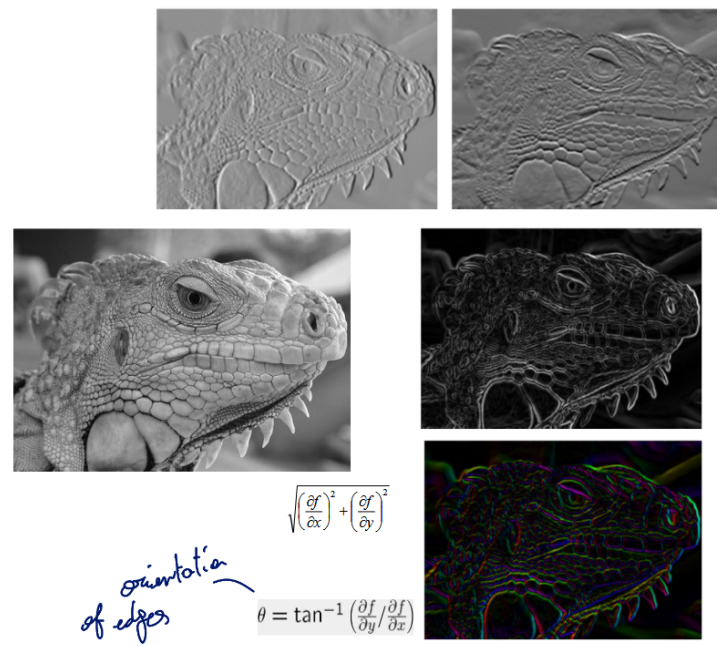


Figure 36: Combining the two (top left: x-dir, top right: y-dir)

It is a great technique to map individual pixel edges, but cannot be perfect to edge detection. There can be gap, several pixels thick, weak or salient edges, and so on. But Sobel masks are the optimal 3 x 3 convolution filters with integer coefficients for step edge detection.

4.3.3. Zero-crossing

First, we want to do edge thinning, to reduce a several pixel thick edge to only one contiguous line. Once we found the gradient, we want to look at the direction giving the maximum gradient and interpolate to get these values. We can also apply the hysteresis thresholding to link up the edges. The idea is to have a high threshold for strong edges and a low threshold for weak edges. We then link the weak edges to the strong ones if they are connected.

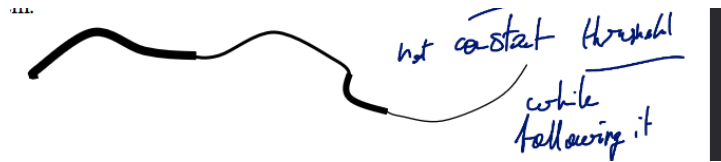


Figure 37: Canny edges

To do zero-crossing detection, we need to use second order derivative and make it equal to 0 to find inflexion points. This is linear + shift invariant (convolution) and isotropic (equal contribution of x and y). The Laplacian is sensitive to noise, so we often add a smoothing step (Gaussian) before applying the Laplacian. This is called the Laplacian of Gaussian (LoG) operator.

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} = 0$$

We can also use the Difference of Gaussian (DoG) which is a good approximation of the LoG and is much faster to compute. The idea is to take the difference between two Gaussian blurred images with different standard deviations.

$$G_1 = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix} \quad G_2 = \begin{pmatrix} 1 & 2 & 1 \\ 2 & -12 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

This DoG method will give us a closed-contour as it is based on the second order derivative. The zero-crossing will give us the edge location. The main issue is that it is not very good at linking up edges and can be quite sensitive to noise.

4.3.4. Harris corner detection

This method will distinguish between homogeneous areas, edges and corners. The idea is to look at the eigenvalues of the second moment matrix of the gradient. If both eigenvalues are small, we have a homogeneous area. If one eigenvalue is large and the other is small, we have an edge. If both eigenvalues are large, we have a corner. The second order moment matrix is given by:

$$\begin{pmatrix} \left(\frac{\partial f}{\partial x}\right)^2 & \frac{\partial f}{\partial x} \frac{\partial f}{\partial y} \\ \frac{\partial f}{\partial x} \frac{\partial f}{\partial y} & \left(\frac{\partial f}{\partial y}\right)^2 \end{pmatrix}$$

looking through a small window, then shifting a window in *any direction* should give a *large change* in intensity

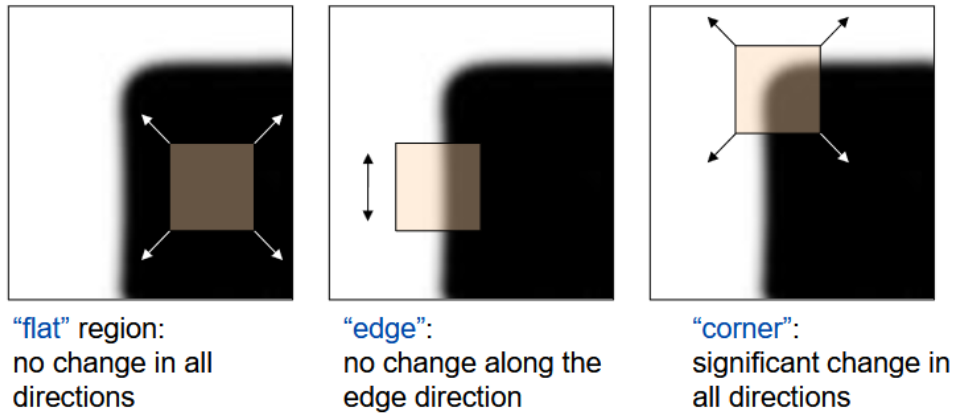


Figure 38: Principle of Harris

The idea is to find the directions of minimal and maximal change in the image. We move the kernel by a small amount (u,v) where our w window function can be either a box or a Gaussian. We then look at the change in intensity which is given by:

$$E(u, v) = \sum_{x,y} w(x, y) [f(x + u, y + v) - f(x, y)]^2$$

$$E(u, v) \approx [u, v] M \begin{bmatrix} u \\ v \end{bmatrix} \quad \text{with } M = \sum_{x,y} w(x, y) \begin{pmatrix} \left(\frac{\partial f}{\partial x}\right)^2 & \frac{\partial f}{\partial x} \frac{\partial f}{\partial y} \\ \frac{\partial f}{\partial x} \frac{\partial f}{\partial y} & \left(\frac{\partial f}{\partial y}\right)^2 \end{pmatrix}$$

To analyze the harris detector, we use the iso-lines $= \det - k \text{trace}^2 = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$. Where λ_1 and λ_2 are the eigenvalues of the matrix M and represent a change in one or the other direction. If the iso-line is positive, we have a corner. If it is negative, we have an edge. If it is zero, we have a homogeneous area. The main issue with this method is that it is not very good at distinguishing between edges and corners in the presence of noise.

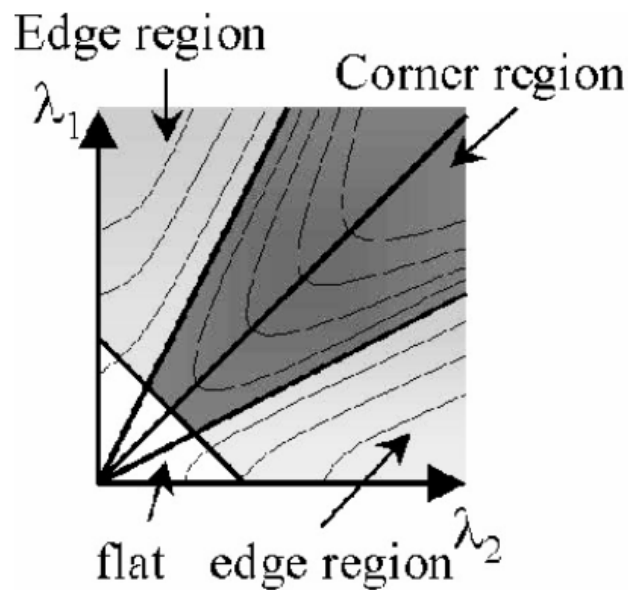


Figure 39: Iso-lines for Harris

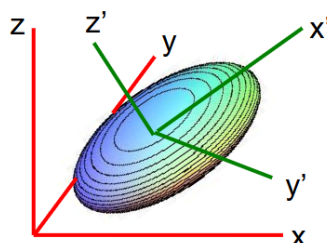
5. Image decomposition

Images can be projected in various orthogonal basis, (pixels, fourier domain, ...). We can also do a rotation for example, but the most important thing is that we use **unitary transformations** $\rightarrow U^*U = I$.

5.1. PCA

We want to reduce the dimensionality of the data while preserving as much variance as possible. We can do this by projecting the data onto the eigenvectors of the covariance matrix. The eigenvectors with the largest eigenvalues will be the principal components. This is a linear transformation that preserves the distances between points in the original space. It is an **unitary transformation** but **data dependent**. We want to reduce the dimension of the data while keeping as much information as possible.

Central idea:



Instead of expressing the data in the original, red, coordinate system, we will instead use the green coordinate system, and center the data around the center of gravity

Figure 40: Central idea of PCA

The Shannon entropy $H(x)$ quantifies the amount of entropy in a data set with a distribution P . This fact is interesting, because most pictures only span a sub-set of the total possible images. For example, natural images have a lot of structure and regularity, which means that they have a low entropy compared to random noise. This is why we can use PCA to reduce the dimensionality of the data while preserving most of the information.

$$H(x) = - \sum_i P(x_i) \log P(x_i)$$

This issue with the lack of spanning in high-dimensional space is often referred to as the **curse of dimensionality**. PCA is a solution to this problem. Some intuitive explanation: 1D: $(r + \delta)/r \approx 1 + \delta/r$; 2D: $(r + \delta)^2/r^2 \approx 1 + 2\delta/r$; D: $(r + \delta)^D/r^D \approx 1 + D\delta/r$. So the volume of the space grows exponentially with the dimension, which means that we need an exponentially large number of samples to cover the space. PCA helps us to reduce the dimensionality of the data while preserving most of the information.

For the PCA, we use the Pearson correlation coefficient. We need to be careful with it as it only indicates linear correlation between two variables!

$$p_{ij} = \frac{\text{cov}(x, y)}{\sigma_i \sigma_j} = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^N (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^N (y_i - \bar{y})^2}}$$

We first find the highest component with the maximum of correlation, then we find the second one with the lowest correlation. We can then apply a rotation in a high dimensional space.

By working around the mean, we benefit from parserval-equality which indicates the variances in x_i will be the same in z_j . This indicates a redistribution of the energy.

$$\sum_{i=1}^p \sigma_i^2 = \sum_{j=1}^p \tilde{\sigma}_j^2$$

5.1.1. Method

With x a vector of p variables. We first look for a linear combination $c_1^\top x$ which has the maximum variance. We then look for a second linear combination $c_2^\top x$ which has the maximum variance and is uncorrelated with the first one. We repeat this process until we have p linear combinations. The first few linear combinations will capture most of the variance in the data, which is why we can reduce the dimensionality of the data by keeping only the first few principal components.

By relying on the Gaussian distribution postulat, we can derive a covariance matrix. Note, we need to center the data around the mean first. We need to maximize so $\text{var}(c_1^\top x) = \sum c_1^\top x (c_1^\top x)^\top = c_1^\top \sum (xx^\top) c_1 = c_1^\top C c_1$ with C the covariance matrix. And we make sure that $c_1^\top c_1 = 1$ to avoid trivial solution.

$$C = \begin{bmatrix} E[(X_1 - \mu_1)(X_1 - \mu_1)] & E[(X_1 - \mu_1)(X_2 - \mu_2)] & \cdots & E[(X_1 - \mu_1)(X_n - \mu_n)] \\ E[(X_2 - \mu_2)(X_1 - \mu_1)] & E[(X_2 - \mu_2)(X_2 - \mu_2)] & \cdots & E[(X_2 - \mu_2)(X_n - \mu_n)] \\ \vdots & \vdots & \ddots & \vdots \\ E[(X_n - \mu_n)(X_1 - \mu_1)] & E[(X_n - \mu_n)(X_2 - \mu_2)] & \cdots & E[(X_n - \mu_n)(X_n - \mu_n)] \end{bmatrix}$$

Sample = $(X_1, X_2, X_3, \dots, X_n)$

Figure 41: Covariance matrix

We can then use the Lagrange multiplier with the constrained mentioned earlier:

$$\mathcal{L}(c_1, \lambda) = c_1^T C c_1 - \lambda(c_1^T c_1 - 1)$$

$$\frac{\partial \mathcal{L}(c_1, \lambda)}{\partial c_1} = 2C c_1 - 2\lambda c_1 = 2c_1 \underbrace{(C - \lambda I)}_{\text{eigenvalue problem}} = 0 \quad \frac{\partial \mathcal{L}(c_1, \lambda)}{\partial \lambda} = c_1^T c_1 - 1 = 0$$

So $c_1^T C c_1 = c_1^T \lambda c_1 = \lambda c_1^T c_1 = \lambda$ with λ as large as possible. Decorrelation and orthogonality are the same thing in this case since we are in a Euclidean space. So $c_2^T C c_1 = c_2^T \lambda c_1 = \lambda c_2^T c_1 = 0$.

Proof for $k = 2$

Maximize $c_2^T C c_2$ being uncorrelated with $c_1^T x$

$$\text{cov} [c_1^T x, c_2^T x] =$$

$$c_1^T C c_2 = c_2^T C c_1 = c_2^T \lambda_1 c_1 = \lambda_1 c_2^T c_1 = \lambda_1 c_1^T c_2 = 0$$

Thus uncorrelatedness becomes

$$c_1^T C c_2 = 0, \quad c_2^T C c_1 = 0, \quad c_1^T c_2 = 0, \quad c_2^T c_1 = 0$$

i.e. c_2 is orthogonal to c_1

Figure 42: Proof of orthogonality between c_1 and c_2

We can again use a Lagrange multiplier to ensure decorrelation:

PCA: algorithm

Then, using λ, ϕ as Lagrange multipliers

$$c_2^T C c_2 - \lambda (c_2^T c_2 - 1) - \phi c_2^T c_1$$

Differentiation w.r.t. c_2 gives

$$C c_2 - \lambda c_2 - \phi c_1 = 0$$

Multiplication on the left by c_1^T gives

$$c_1^T C c_2 - \lambda c_1^T c_2 - \phi c_1^T c_1 = 0$$

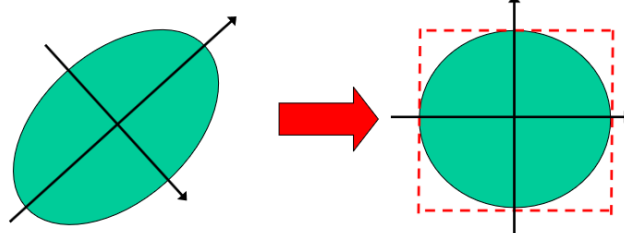
Thus $\phi = 0$

Therefore, $C c_2 - \lambda c_2 = 0$, i.e. $(C - \lambda I_p) c_2 = 0$

Again, $\lambda = c_2^T C c_2$, so select 2nd largest λ

Figure 43: Decorrelation

The decorrelated projections correspond to the main axes of the distribution. These are orthogonal.



Important note: principal components are guaranteed to be independent only if the data set is jointly normally distributed (i.e. ellipsoid-shaped).

Through the covariance matrix we only extract moments up to 2nd degree, and needed to specify such Gaussian distribution

Figure 44: Important remark

5.1.2. Usage

- Classification of crops: using Near-infrared, Red, Green data. We see some correlation between red and green + near-infrared. We can compress the data to roughly 60% while retaining similar performance.
- Culture and heritage: reveal details that wasn't visible before. Quickly find the most important features in the data.
- Image compression: same density in neighboring pixels. Issue is that image of N pixels $= N^2$ base. So $N^2 * N^2$ for the covariance matrix. PCA allows us to reduce the dimensionality of the data while preserving most of the information. We can then use a lossy compression algorithm to further reduce the size of the data.
 - We can use the **eigenfaces** of the covariance matrix as a basis for the image. We can then project the image onto this basis and keep only the first few components to reduce the size of the data. This is called **eigenface**

method. We form a human face base using a base of already existing faces. (do not forget to subtract the mean !)

5.1.3. Limitations

As described before, the Pearson correlation coefficient is only for linear correlation. Moreover, the PCA is based on the assumption of a Gaussian distribution.

A solution is the Interesting Component analysis where we find multiple “interesting” components. Or we can use the Linear Discriminant Analysis (LDA) which is a supervised method that finds the linear combination of features that best separates two or more classes of objects or events. It is based on the idea of maximizing the between-class variance while minimizing the within-class variance. This approach is better for non Gaussian distribution.

There is also the support vector machine which look at an hyper plane to find the best separation between classes.

PCA looks like the Harris corner detector but a few things change:

- No subtraction by mean in Harris
- No normalization in Harris

6. Surface features color and texture

6.1. Color

It is important to distinguish between two important spectre of the color:

1. Radiometry: physical measurement of the light (radiance, irradiance, reflectance, ...)
2. Photometry: how the human eye perceives the light (brightness, colour, ...)

A light l with a spectral composition $c(\lambda)$ and the human that perceives it with a sensitivity $h(\lambda)$, with the constant $k = 683$ lm/W. Ofc, we can span only the human visible spectrum which is roughly between 400 and 700 nm. The perceived brightness is then given by the following equation:

$$l = k \int_{\lambda=0}^{\infty} c(\lambda)h(\lambda)d\lambda$$

Human perception of color can be summarized with Brightness, Huhe, and saturation.

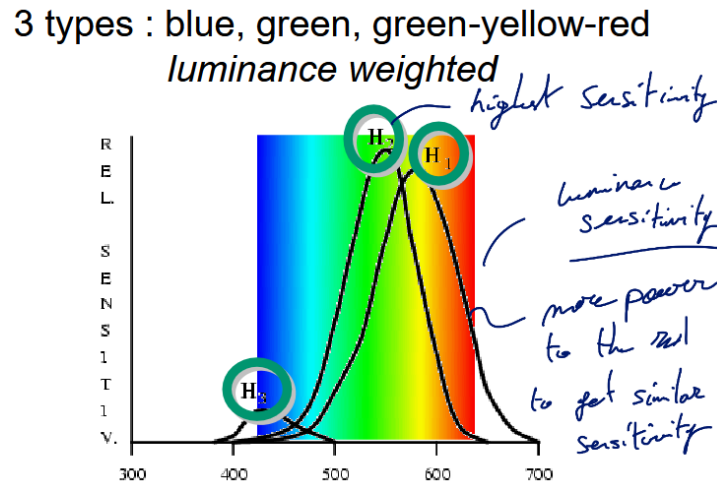


Figure 45: Retinal cones of a human; occupancy S:5-10, M:30, L:60

So the human has 3 sensitivity curves and not just one from the equation aforementioned. We have 3 responses for the full spectrum. So if 2 sources are observed with the same response for the 3 cones, they will be perceived as the same color. This is called **metamerism**.

6.1.1. Commission Internationale de l'Éclairage (CIE)

They standardized the color space with 3 primaries:

1. Red: 700 nm
2. Green: 546.1 nm
3. Blue: 435.8 nm

With this, we can match m_j a source $C(\lambda)$ with a combination of the 3 primaries $P_j(\lambda)$ such that:

$$C(\lambda) = \sum_{j=1}^3 m_j P_j(\lambda)$$

We can derive the following:

$$R_i(C) = \int \sum_{j=1}^3 m_j P_j(\lambda) H_i(\lambda) d\lambda = \sum_{j=1}^3 m_j \underbrace{\int P_j(\lambda) H_i(\lambda) d\lambda}_{l_{i,j} \text{ Sensitivity of a primary can be determined offline}} \quad d\lambda$$

This makes a really simple system of 3 linear equations for the human eye. If we use different primaries L' , we can find a new set of coefficients m' such that:

$$\sum_{j=1}^3 m_j l_{i,j} = R_i \quad LM = R \quad L'M' = R \quad m' = L'^{-1} Lm$$

6.1.1.1. Tristimulus value As white is the combination of all the colors, we can use the white point as a reference for the color. We can then define the tristimulus value as:

$$T_j = \frac{m_j}{w_j} \quad T_1 = T_2 = T_3 = 1 \quad \text{for white}$$

$$R_i(C_\lambda) = H_i(\lambda) = \sum_{j=1}^3 m_j l_{i,j} = \sum_{j=1}^3 w_j l_{i,j} T_j(\lambda)$$

for the CIE primaries:

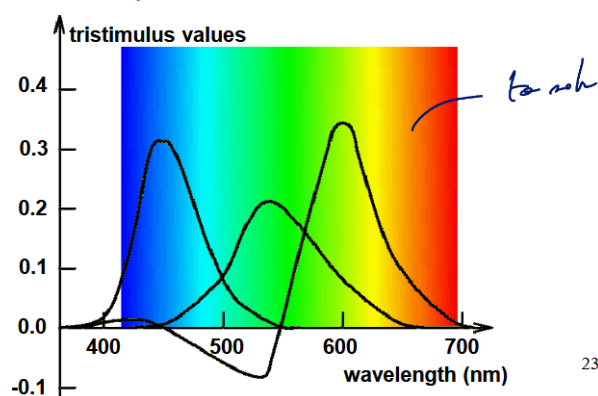


Figure 46: Spectral matching curves

We notice negative colors which cannot be produced by a combination of the primaries. This is a problem for the display of colors. Moreover, for some colors, a triple of primaries cannot represent it! However the tristimulus values still contain brightness information, we can normalize to only get chromatic information:

$$t_j = \frac{T_j}{\sum_{j=1}^3 T_j} \quad t_1 + t_2 + t_3 = 1$$

So now, 2 chromaticity coordinates are enough to represent the color (saturation and hue). This forms the XYZ coordinates of the CIE, with X being a mix of red/green cones, the Y the luminance and Z blue light primarily. The mapping matrix is

$$\begin{pmatrix} X \\ Y \\ Z \end{pmatrix} = \begin{pmatrix} 0.490 & 0.310 & 0.200 \\ 0.177 & 0.813 & 0.011 \\ 0.000 & 0.010 & 0.990 \end{pmatrix} \begin{pmatrix} R \\ G \\ B \end{pmatrix}$$

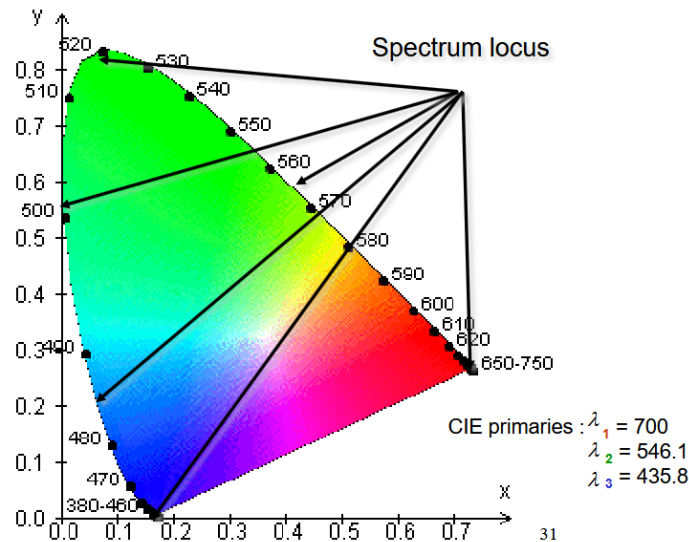


Figure 47: CIE color triangle

Inside this triangle, the mix of two colors in some quantities lie on the straight line between those two colors.

Screen standard tries to span as much as this CIE triangle to have the highest fidelity possible. Such color coordinates need 3 primaries on the CIE triangle + the central white point → 4 points. Also the projection is skewed on the triangle, much more information lie in the bottom blue corner and red corner than in the top green one.

The CIE u-v is a solution which represents area more faithfully.

$$u = \frac{4x}{-2x + 12y + 3} \quad v = \frac{6y}{-2x + 12y + 3}$$

6.1.2. Constancy

Many optical illusions exists where our brain is tricked and interprets colors when there are none or the context can influences our perception. We often interpret the ambient light to infer the result of the dot product on our human cones. This is called the **color constancy**. The brain is trying to infer the color of the object by taking into account the ambient light. This is why we can see a white paper as white even if it is under a red light.

6.2. Texture

Many textures exist such as oriented vs isotropic, stochastic vs regular, coarse vs fine, ...

6.2.1. Fourier features

We analyze the texture based on the power spectrum of the Fourier transform. A peak in the power spectrum indicates a strong periodicity in the image. The orientation of the peak indicates the orientation of the texture. The width of the peak indicates the coarseness of the texture. The more spread out the peak is, the more stochastic the texture is.

One problem with this method is the fact that it captures the global periodicity of the image but not the local one. Moreover, it is not very good at capturing the orientation of the texture. Finally, it is not very good at capturing the coarseness of the texture.

$$\int \int |F(u,v)|^2 du dv$$

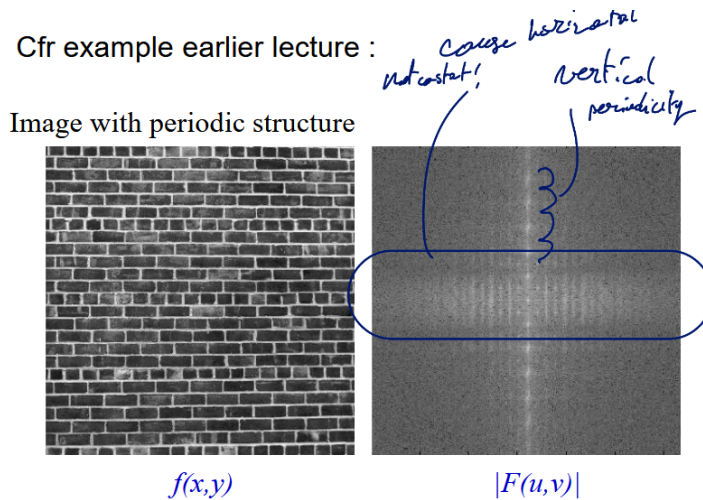


Figure 48: Periodicity

6.2.2. Cooccurrence matrices

It is inspired by the histogram idea, but this time we will take a chosen offset between two points and count how many i,j pair exists in a double entry matrix. This will give us a measure of the texture based on the co-occurrence of pixel values. We can then derive some statistics from this matrix such as contrast, homogeneity, energy, ...

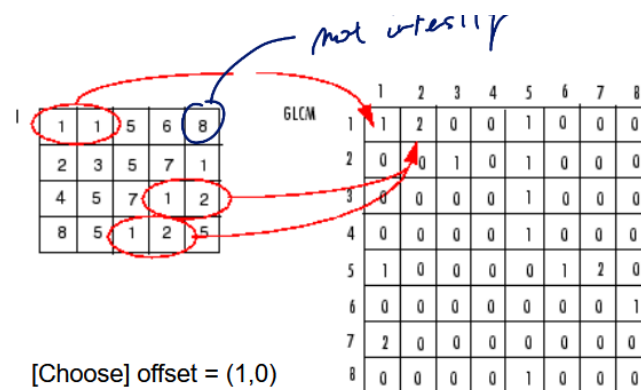


Figure 49: Cooccurrence matrix

Table 6: Summary table of the features derived from the cooccurrence matrix

Feature	Expression
Contrast	$\sum_{i,j} (i - j)^2 C(i, j)$
Homogeneity	$\sum_{i,j} \frac{1}{1 + \ i - j\ } C(i, j)$

Feature	Expression
Energy	$\sum_{i,j} C(i,j)^2$
Entropy	$-\sum_{i,j} C(i,j) \log C(i,j)$
Max. Probability	$\max_{i,j} C(i,j)$

6.2.3. Filter banks

Feature	1D filter	
L3	[1 2 1]	gauss perim wav filter
E3	[-1 0 1]	
S3	[-1 2 -1]	
L5	[1 4 6 4 1]	L a w s f i l t e r s
E5	[-1 -2 0 2 1]	
S5	[-1 0 2 0 -1]	
W5	[-1 2 0 -2 1]	
R5	[1 -4 6 -4 1]	
L7	[1 6 15 20 15 6 1]	L a w s f i l t e r s
E7	[-1 -4 -5 0 5 4 1]	
S7	[-1 -2 1 4 1 -2 -1]	
W7	[-1 0 3 0 -3 0 1]	
R7	[1 -2 -1 4 -1 -2 1]	
O7	[-1 6 -15 20 -15 6 -1]	

76

Figure 50: Laws filter

6.2.3.1. Laws We can start from 1D laws filter to form more advanced decomposed 2D filters. L=Level, E=Edge, S=Spot, W=Wave, R=Ripple. The idea is to have a set of filters that can capture different types of textures. For example, the L5E5 filter will capture edges, while the E5L5 filter will capture spots. We can then apply these filters to the image and analyze the response to get a measure of the texture. It is simple but really effective. Detection or classification nby assigning features / statistical metrics to the outputs.

6.2.3.2. Gabor It is a gaussian envelope modulated by a sinusoidal plane wave. It is a band-pass filter that can capture both the frequency and orientation of the texture. We can tweak the sensitivity and directionality of the filter. The Gabor filter is defined as:

$$g(x, y) = \exp\left(-\frac{x^2 + \gamma^2 y^2}{4\Delta_{x,y}^2}\right) \cos(2\pi u^* x + \phi) \longleftrightarrow G(u, v) = \frac{1}{4\pi\Delta_{u,v}^2} \exp\left(-\frac{(u - u^*)^2 + v^2}{4\Delta_{u,v}^2}\right) + \exp\left(-\frac{(u + u^*)^2 + v^2}{4\Delta_{u,v}^2}\right)$$

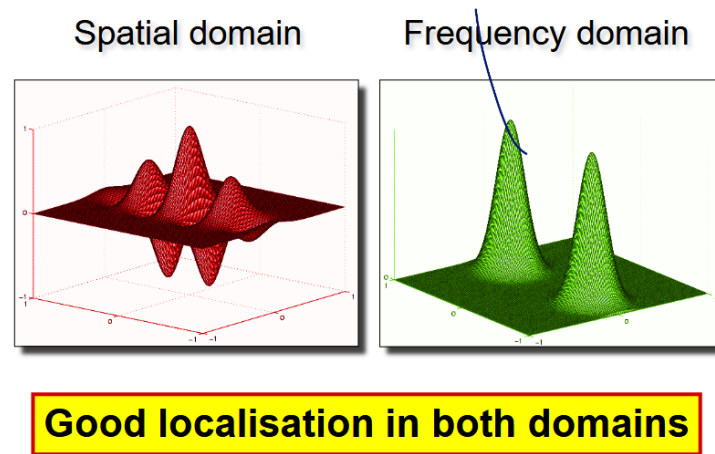


Figure 51: Gabor filters

6.2.3.3. Eigenfilters

1. Create a mask to be shifted over a training image
2. Within the mask, collect intensity statistics
3. PCA → eigenvectors → eigenfilters
4. Determine energies of eigenfilters of eigenfilter outputs

We often need large but sparse filters for good efficacy. Can be great for damage inspection or QA in industry.

We often need to use multiple eigenfilters to capture different aspects of the texture. A great technique to compile multiple eigenfilters together and quickly locate error or irregularities in the texture is the **Manhattan distance of the energies**:

$$D_M(x, y) = \sqrt{(r(x, y) - \mu)^T \Sigma^{-1} (r(x, y) - \mu)} \quad (9)$$

$$\mu = \frac{1}{N} \sum_{i=1}^N r(x_i, y_i) \quad \Sigma = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_K) \quad (10)$$

Mahalanobis distance DM measures how unusual this combination of responses is compared to normal texture. A response along a very stable direction (small eigenvalue) counts more. A response along a highly variable direction (large eigenvalue) is less surprising.

7. Motion Extraction

Many animals are dependent on motion detection to outperform camouflage. Only with a few dots in motion can infer many informations.

7.1. Definition

7.1.1. Optical flow

Optical flow

Apparent motion of brightness patterns in the image. Ideally it is the the projection of the 3D motion vectors of the image

Motion Field

3D motion of the scene as seen through the camera. It is the projection of the 3D motion vectors of the scene onto the image plane.

We can use 2D vectors to depict the translation of the pixels forming the optical flow. Motion field != optical flow (e.g., the barbershop pole turning around horizontally but it looks like it's sliding vertically, same for a perfect sphere with an uniform texture, it will look stable even if it starts spinning)

The goal with optical flow is to measure for each pixel between two frames where each went. It is a non-trivial task and requires a few assumptions to be achievable:

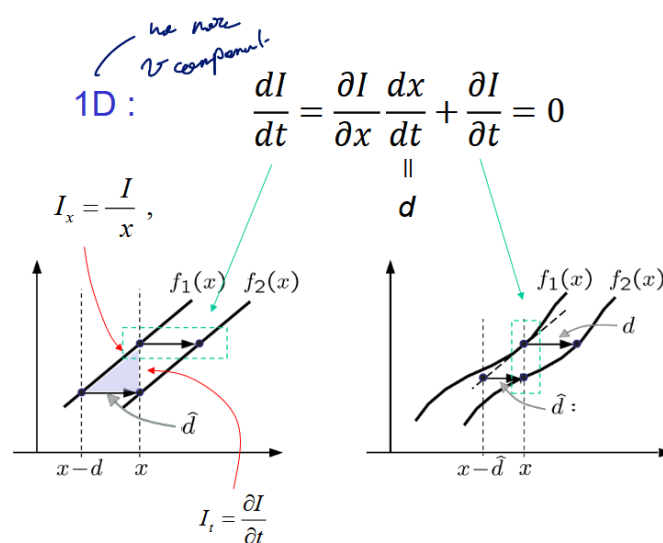
1. Same brightness
2. Minimal motion

7.1.1.1. Mathematics With $I(x, y, t)$ the intensity of the pixel at position (x, y) at time t , we can measure the difference with:

$$I(x, y, t) = I(x + \delta x, y + \delta y, t + \delta t) \quad \frac{dI}{dt} = \underbrace{\frac{\partial I}{\partial x}}_{I_x} \underbrace{\frac{dx}{dt}}_u + \underbrace{\frac{\partial I}{\partial y}}_{I_y} \underbrace{\frac{dy}{dt}}_{vD} + \underbrace{\frac{\partial I}{\partial t}}_{I_t} = 0$$

This equation states that if one were to track the image projections of a scene point through the video, it would not change its intensity. This tends to be true over short lapses of time.

Note that dI/dt represents the change of intensity when following a physical point through the images; but $\partial I/\partial t$ is the change of intensity at a fixed location in the image. The difference between these two is the motion of the point in the image.



35

Figure 52: Optical flow in 1D

The issue with this equation is that I_x, I_y , and I_t can be measured. However, u and v are unknown, 1 equation 2 unknowns = **Aperture Problem**.

Aperture problem : only the component along the gradient can be retrieved

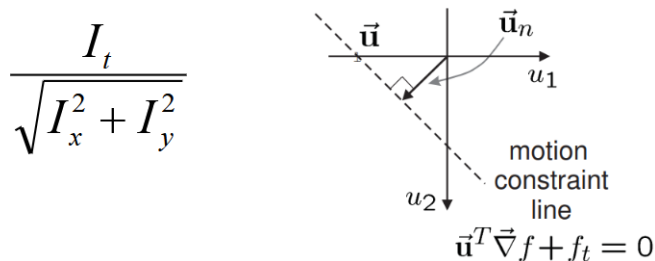


Figure 53: Aperture problem

We only zoom in one tiny part of the frame and thus we miss the overall motion by only looking at such details. The problem can be solved with:

1. Higher derivative of intensity $\rightarrow$ expensive + noisy
2. For planar intensity profile, problem cannot be solved (higher order derivative are zero)

7.2. Horn & Schunk

Additional smoothness constraint is added to the equation to solve the aperture problem. The idea is to minimize the following energy function:

$$e_S = \int \int ((u_x^2 + u_y^2 + v_x^2 + v_y^2) dx dy) \quad e_C = \int \int (I_x u + I_y v + I_t)^2 dx dy \quad \text{minimize } e = e_S + \lambda e_C$$

We need to find a function that extremize the functionals (functions that takes vector in and scalar out)

look for functions that extremize **functionals**

$$I = \int_{x_1}^{x_2} F(x, f, f') dx \quad \text{with } f = f(x), f' = \frac{df}{dx}$$

Functional (pointing to F)

limit of image (under the integral limits)

$$f(x_1) = f_1 \quad \text{and} \quad f(x_2) = f_2$$

For instance, in **1D**: $f = u, f' = u_x$

$$F(x, u, u') = (u_x^2 + u_y^2) + (v_x^2 + v_y^2) + \lambda(I_x u + I_y v + I_t)^2$$

Figure 54: 1D example of our functional

Suppose

1. $f(x)$ is a solution
2. $\eta(x)$ is a test function with $\eta(x_1) = 0$ and $\eta(x_2) = 0 \rightarrow$ boundary conditions

We then consider

$$I = \int_{x_1}^{x_2} F(x, f, f') dx \quad \text{with } f = f(x), f' = \frac{df}{dx}$$

$$I = \int_{x_1}^{x_2} F(x, f + \varepsilon \eta, f' + \varepsilon \eta') dx \quad \leftarrow f \rightarrow f + \varepsilon \eta$$

→ not 0 causing → looking for a complete function!

Rationale: supposed f is the solution, then any deviation should result in a worse I ; when applying classical optimization over the values of ε the optimum should be $\varepsilon = 0$

Figure 55: Calculus of variation

This allows to reformulate an optimization over a function into an optimization over a scalar. We can use $dI/d\varepsilon|_{\varepsilon=0} = 0$. We can use Euler-Lagrange equation to find the function that extremize the functional.

TODO: PAGES 49 - 62 → Take time to understand

1. We use regularization, need sometimes to pose extra constraints
2. Errors at object boundaries, smoothness constraint no longer valid

7.3. Lucas & Kanade

It uses the same optical flow equation but used an area based regression. We assume that the flow is constant in a small neighborhood of the pixel under consideration (local optimization). We can then solve for the flow vector using least squares. The idea is to minimize the following energy function:

$$A = \begin{bmatrix} I_x(q_1) & I_y(q_1) \\ I_x(q_2) & I_y(q_2) \\ \vdots & \vdots \\ I_x(q_n) & I_y(q_n) \end{bmatrix} \quad v = \begin{bmatrix} V_x \\ V_y \end{bmatrix} \quad b = \begin{bmatrix} -I_t(q_1) \\ -I_t(q_2) \\ \vdots \\ -I_t(q_n) \end{bmatrix} \quad (11)$$

$$A^T A v = A^T b \quad v = (A^T A)^{-1} A^T b$$

$$\begin{bmatrix} V_x \\ V_y \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^n I_x(q_i)^2 & \sum_{i=1}^n I_x(q_i) I_y(q_i) \\ \sum_{i=1}^n I_x(q_i) I_y(q_i) & \sum_{i=1}^n I_y(q_i)^2 \end{bmatrix}^{-1} \begin{bmatrix} -\sum_{i=1}^n I_x(q_i) I_t(q_i) \\ -\sum_{i=1}^n I_y(q_i) I_t(q_i) \end{bmatrix}$$

It cannot always reliably calculate the flow, aperture problem, it requires a lot of texture in all direction to be reliable. So, to detect interesting points, we often use the Harris-detector to find interesting spots and apply area based flow around

each key point. This is called the **feature tracking**. However, it does not respect Taylor expansion.

Another technique is to use a form of coarse to fine to estimate the flow even better with various size.

7.4. Active contours

The idea is to track a contour from one key frame to another one. It is like an elastic band we adjust frame by frame. It is a good and robust approach for noise especially when we know what object we are tracking (e.g., lips).

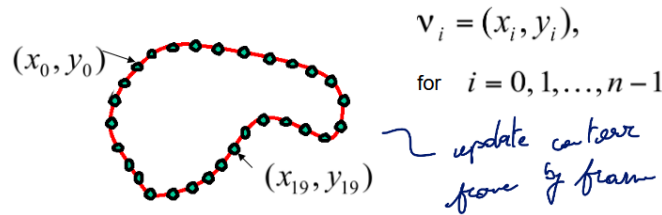


Figure 56: Active contours

We use some discrete representation of the contour which is a list of 2D points (vertices). We define an energy function that we want to minimize. The energy function is a combination of an internal energy that depends on the shape of the contour (known shape, ...) and an external energy that depends on the image (edges present in the image, attracts the contour to the edges of the image). The internal energy is given by:

$$E_{total} = E_{internal} + E_{external}$$

7.4.0.1. External Energy for a point of a curve and the whole curve:

$$E_{external}(v) = -(|G_x(v)|^2 + |G_y(v)|^2) \quad E_{external} = - \sum_{i=0}^{n-1} (|G_x(x_i, y_i)|^2 + |G_y(x_i, y_i)|^2)$$

7.4.0.2. Internal We use the elasticity and the bending energy to define the internal energy. The elasticity energy is given by:

$$E_{internal}(v(s)) = \underbrace{\alpha \left| \frac{dv}{ds} \right|^2}_{\text{elasticity}} + \underbrace{\beta \left| \frac{d^2v}{ds^2} \right|^2}_{\text{bending}}$$

For our discrete representation, we can use finite differences to approximate the derivatives:

$$\frac{dv}{ds} \approx v_{i+1} - v_i \quad \frac{d^2v}{ds^2} \approx (v_{i+1} - v_i) - (v_i - v_{i-1}) = v_{i+1} - 2v_i + v_{i-1}$$

Internal energy for the whole curve

$$E_{internal} = \sum_{i=0}^{n-1} (\alpha |v_{i+1} - v_i|^2 + \beta |v_{i+1} - 2v_i + v_{i-1}|^2)$$

For the elastic energy, we often add a term d to avoid that the energy collapse on itself and so we take the initial distance d .

We can also use a term to penalize the length of the curve to avoid that it grows indefinitely. This is called the balloon energy.

e.g., α weight controls the penalty for internal elasticity

$$E_{total} = E_{internal} + \gamma E_{external}$$

$$E_{internal} = \sum_{i=0}^{n-1} \alpha \left(\bar{d} - \|v_{i+1} - v_i\| \right)^2 + \beta \|v_{i+1} - 2v_i + v_{i-1}\|^2$$

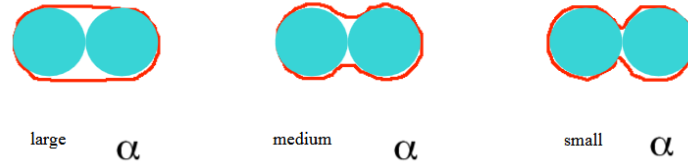


Figure 57: Active contours

7.4.1. Energy minimization

7.4.1.1. Greedy For each point we simply look where we can move them to minimize the energy. Usually small window (5x5) around the point. It is simple but can get stuck in local minima and requires decent initialization.

7.4.1.2. Dynamic programming Use a form of the viterbi algorithm to propagate down the energy and find the optimal path. It is more robust to local minima but can be computationally expensive.

$$\iint \mu (u_x^2 + u_y^2 + v_x^2 + v_y^2) + \|\nabla f\|^2 \|\mathbf{g} - \nabla f\|^2 dx dy$$

At minimum, Euler-Lagrange equations must hold

$$\begin{aligned} \mu \Delta u - (u - f_x) (f_x^2 + f_y^2) &= 0, \\ \mu \Delta v - (v - f_y) (f_x^2 + f_y^2) &= 0, \end{aligned}$$

Solved by gradient descent / time evolution:

$$\begin{aligned} u_t(x, y, t) &= \mu \Delta u(x, y, t) - (u(x, y, t) - f_x(x, y)) (f_x(x, y)^2 + f_y(x, y)^2) \\ v_t(x, y, t) &= \mu \Delta v(x, y, t) - (v(x, y, t) - f_y(x, y)) (f_x(x, y)^2 + f_y(x, y)^2) \end{aligned}$$



Figure 58: GVF (Informational only)

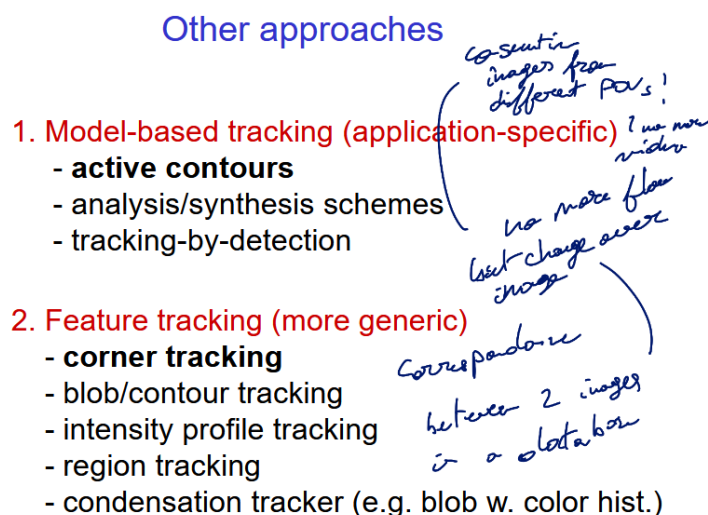


Figure 59: Other approaches

7.4.1.3. Gradient Vector flow (GVF)

8. Local features & Feature matching

The goal is to recognize landmarks across pictures, or faces, ... It is based on previous knowledge heavily inspired from Sobel, Harris Laplacians, Lucas Kanade (for feature tracking), ... All of this will help us build SIFT.

We want to find local features but it is not trivial as scales will be problematic in this task. But achieving it will enable 3D reconstruction, recognition of objects, image matching, image retrieval, ... For this we rely on 3 main components:

1. Detection: of the important landmark and interest points
2. Description: of the local features around the interest points (feature vector)
3. Matching: of the features between different images

We want our detector to be:

- repeatable: be consistent
- distinctive: highlight good interest points. Moreover it should precisely match each interest point
- invariant: no matter the scale or the rotation, we should be able to find the same interest points (geometric or photometric changes). Keep deriving the same feature points throughout the images (otherwise impossible to match them)
- compact / efficient: realtime applications, embedded systems, ... with limited compute power, use binary feature descriptors
- find local interest points: the smaller, the less likely it will be affected by occlusion, deformation, ... we can approximate local surfaces using local planes

Everything will be a tradeoff between these different properties. For example, we can have a very distinctive feature that is not very repeatable, or a very repeatable feature that is not very distinctive. We need to find the right balance between these properties for our specific application.

8.1. Detection

8.1.1. Rotation invariant detectors

8.1.1.1. Harris corner points We can use the harris detector to detect interesting feature points such as corners. When selecting the point, the window encapsulating it should make it recognizable and if we slide around we should witness a large change in intensity. M is symmetric since we have change in both direction (corner) and we have

$$M = \Sigma \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} \quad Mx_i = \lambda_i x_i \quad R(x, y) = \det(M) - \alpha \text{tr}(M)^2 = \lambda_1 \lambda_2 - \alpha(\lambda_1 + \lambda_2)^2$$

We get the R scores for each image window, find points where surrounding window gave high R , finally take the points of local maxima of R .

- Second moment matrix / autocorrelation matrix

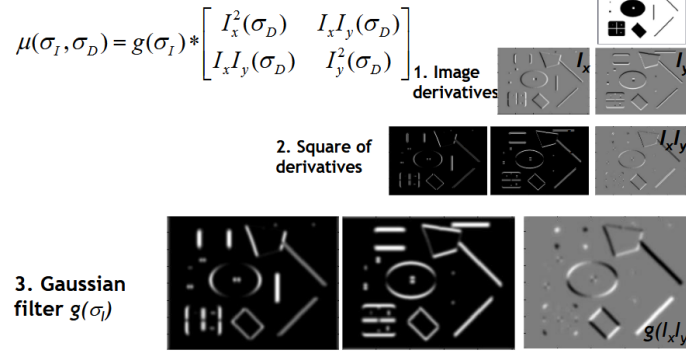


Figure 60: Harris detector in practice

We develop a “cornerness” function where both eigenvalues are large (high det).

$$har = \det[\mu(\sigma_I, \sigma_D)] - \alpha \cdot \text{trace}[\mu(\sigma_I, \sigma_D)] = g(I_x^2)g(I_y^2) - g(I_x I_y)^2 - \alpha(g(I_x^2) + g(I_y^2))^2$$

It is rotation invariant $M = X \Sigma X^{-1}$ with X the rotation matrix. However, it is not scale invariant as the size of the window will change with the scale. Moreover, it is not very good at detecting corners in low contrast images.

8.1.1.2. Laplacian of Gaussians Laplacian of Gaussians (LoG) is a second order derivative operator that is used to detect edges and corners in images. It is defined as the convolution of the image with a Gaussian kernel followed by the Laplacian operator. The idea is to find the zero-crossings of the LoG response, which correspond to edges and corners in the image. It is rotation invariant but not scale invariant as the size of the Gaussian kernel will change with the scale.

Important to match the size of the kernel to have an optimum detection.

8.1.2. Scale invariant detectors

8.1.2.1. Multi-scale approach We can use the Harris or LoG where we will try multiple size of kernel. We can then look for local maxima in the scale space. This is called the **scale-space representation**. The idea is to create a pyramid of

images with different scales and look for local maxima in this pyramid. This is a simple but effective way to achieve scale invariance.

It is important to use a scale-normalized derivatives to achieve scale invariance. This is done by multiplying the derivatives by a factor of σ^n where n is the order of the derivative and σ is the scale. This way, the response of the detector will be the same regardless of the scale.

8.1.2.2. Local extrema in scale space We find the two highest response and then try to match them together. We can then look for local maxima in the scale space to find the interest points. This is called the **scale-space extrema**. The idea is to find the points that are local maxima in both space and scale. This is a more robust way to find interest points as it takes into account the scale of the features.

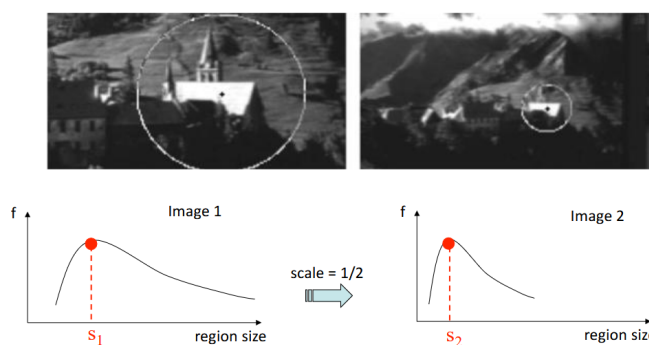


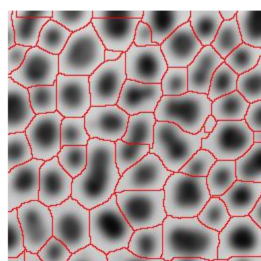
Figure 61: Visual explanation

8.1.3. Affine invariant detectors

The idea is to take advantage of regular human-made plane structure to inverse the affine transformation through projective transformation: **homography**. We can then apply the Harris or LoG detector on the rectified image to find the interest points. This is called the **affine-invariant detectors**. The idea is to find the points that are invariant to affine transformations. This is a more robust way to find interest points as it takes into account the affine transformations that can occur in the image. For local features, this behavior is less important.

Maximally Stable Extremal Regions (MSERs)

- Based on **watershed** algorithm



*first
not in
scale
space*

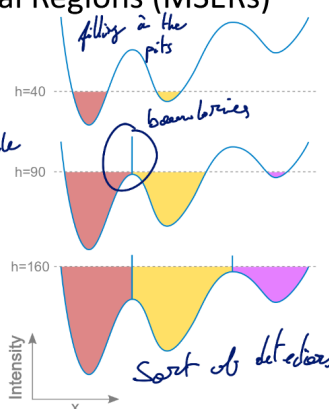


Figure 62: Maximally Stable Extremal Regions

8.1.3.1. MSER We “flood” the area until the change in area is too large. We can then look for local maxima in the scale space to find the interest points. This is called the **maximally stable extremal regions (MSER)**. The idea is to find the points that are local maxima in both space and scale.

Another improvement, we can use ellipsoid instead of weird grainy boundaries, this makes matching easier.

8.2. Description

8.2.1. Scale Invariant Feature Transform (SIFT)

Handle changes in viewpoint up to about 60 degree out of plane rotation. Can also handle changes in illumination, partial occlusion, ... We want to find a descriptor that is invariant to these changes. We can use the local image patch around the interest point to create a descriptor. The idea is to create a histogram of oriented gradients (HOG) around the interest point. This is called the **SIFT descriptor**. It has two steps: rotation normalization and then descriptor computation.

8.2.1.1. 1. Rotation normalization

- take window around detected interest point (3σ)
- Compute edge orientation (gradient angle - 90) for each pixel
- Histogram of edge orientation with 36 bins (10 degree each)
 - Weight by grad magnitude, bilinear interpolation, distance to center, ...
 - * bilinear interpolation: to find an unknown point in a 2D space located inside a square. We take the 4 points of the square and we weight them by the area covered between the point and the unknown. The closer the point is, the more weight it has. This allows us to have a smooth transition between the bins of the histogram.
- Find the dominant rotation and rotate the patch to make it horizontal pointing to the right.

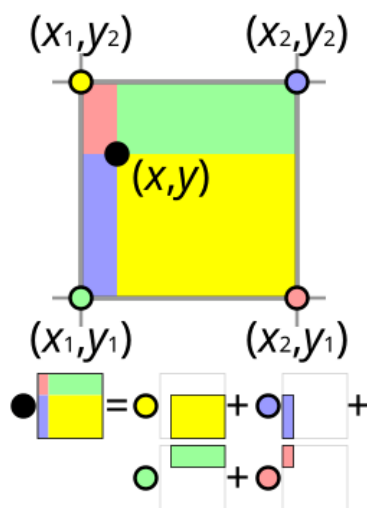


Figure 63: Bilinear interpolation (wikipedia)

8.2.1.2. 2. Descriptor computation

- Divide the square patch into 4x4 sub-regions
- For each sub-region, compute a histogram of edge orientation with 8 bins (45 degree each)

– $16 \times 8 = 128$ dimensions for the descriptor

- Normalize the descriptor to have unit length to achieve invariance to changes in illumination.

It is a robust descriptor because of the use of histograms, bilinear weighting, smoothing with gaussian kernel. However it is not invariant to affine transformations, it is not very good at handling changes in scale, and it is not very good at handling changes in viewpoint. It is good for small geometric transformations and robust for photometric changes.

We often combine the SIFT descriptor with other method or even CNN.

8.3. Matching

As the name implies, the goal is to determine correspondences between features in different images. We can use the Euclidean distance between the descriptors to find the closest match. Another approach is the **Sum of Squared Differences (SSD)**.

However, this can lead to many false matches. To avoid this, we can use a ratio test where we compare the distance of the closest match to the distance of the second closest match. If the ratio is below a certain threshold, we consider it a good match. This is called the **Lowe's ratio test**.

In general we take one feature in an image and use a defined distance function to test all the features in the second and find the minimum distance.

9. 3D Acquisition

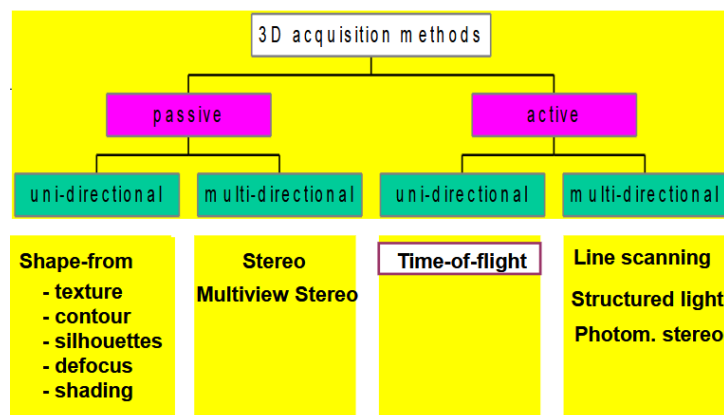


Figure 64: Map of the chapter

9.1. Passive: no interaction with the scene

9.1.1. Uni-directional

9.1.1.1. Shape from texture By using a repetitive pattern (e.g. checkerboard) it is possible to infer the 3D structure of the scene by looking at the orientation of the surface. If an isotropic pattern creates an anisotropic image statistics, we can invert this operation.

9.1.1.2. Shape from contour We make assumption about the contour shape, we can use the symmetry of an object to infer its 3D structure. For example, if we see a circle in the image, we can infer that it is a sphere in 3D. If we see an ellipse, we can infer that it is an ellipsoid in 3D. We can also use the compactness to infer the 3D structure.

9.1.1.3. Shape from silhouette By using multiple points of view, we can create a voxel in 3D to reconstruct in 3D the object.

9.1.1.4. Shape from defocus To infer depth in a picture, we look at the sharpness of the image at various zoom focus to reconstruct the 3D scene. We can create alpha map with this technique.

9.1.1.5. Shape from shading We can use directional lightning (often known direction) to evaluate the depth of the image. By using the Lambertian assumption (reflection depend on an angle; a surface scatters incoming light uniformly in all directions, equally bright in all direction, ideal matte material), we can find correspondance using reflectance maps.

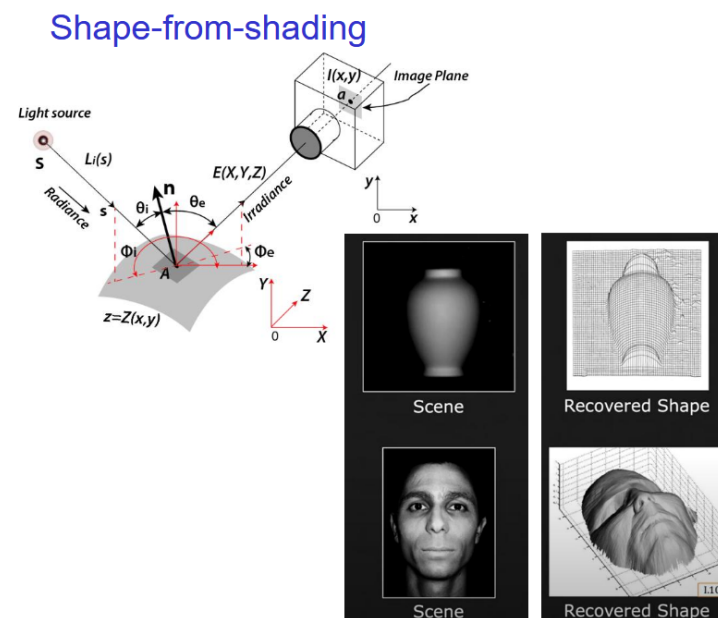


Figure 65: Shape from shading

9.1.2. Multi-directional

9.1.2.1. Basic Stereo vision The idea is to use triangulation like we do with our eyes. By using two cameras with a known baseline, we can find the depth of the scene by looking at the disparity between the two images. The disparity is the difference in the position of the same point in the two images

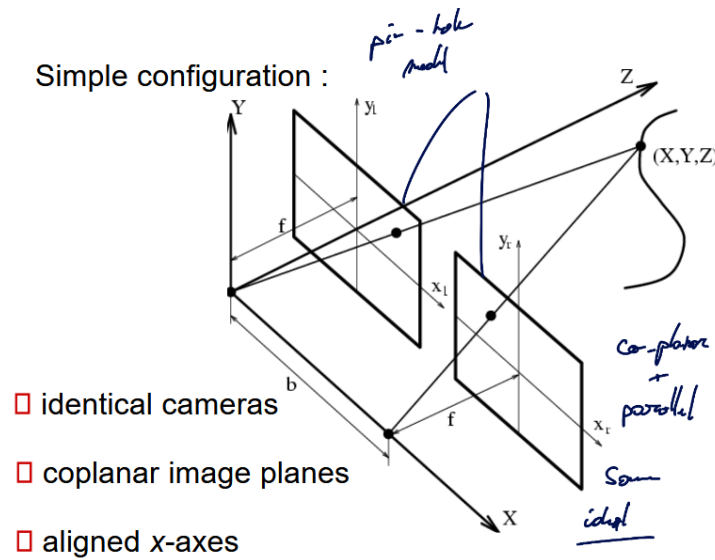


Figure 66: Passive stereo

With this basic setup, the object will lie on the same scanline in both images but the vertical line will be somewhat different. Remember the camera projection with the K camera calibration matrix, R the rotation matrix, P the 3D point and C the camera center:

$$\rho p = KR^T(P - C)$$

If we “attach” the world coordinate to the left camera, we can remove R since $= I$ and $C = 0$. With a distance b separating the two cameras and by normalizing the focal length to 1, we can find the depth Z of the point in the scene with:

$$\rho \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = K \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \quad \rho \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = K \begin{pmatrix} X - b \\ Y \\ Z \end{pmatrix} \quad K = \begin{pmatrix} fk_x & 0 & 0 \\ 0 & fk_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad Z = \frac{fbk_x}{x - x'}$$

$$\begin{cases} x = \frac{fk_x X}{Z} \\ y = \frac{fk_y Y}{Z} \end{cases} \quad \begin{cases} x' = \frac{fk_x (X - b)}{Z} \\ y' = \frac{fk_y Y}{Z} \end{cases} \quad (12)$$

k_x is required for non-square pixels which is 1/horizontal pixel size and k_y is 1/vertical pixel size. $k_x = k_y = 1$ for square pixels. The depth is inversely proportional to the disparity $x - x'$. The larger the disparity, the closer the object is to the camera. The smaller the disparity, the farther the object is from the camera.

When observing the same points in two view, using the **disparity** ($x - x'$), imprecise for far away objects (increase b or f to improve accuracy, but it reduces the simultaneous visibility) we have:

$$X = b \frac{x}{(x - x')} \quad Y = b \frac{k_x}{k_y} \frac{y}{(x - x')} \quad Z = bk_x \frac{f}{(x - x')}$$

Human stereo vision is good only up to ± 10 meters, the human brain will use experience to infer distance for the rest. The real issue is finding the two right correspondances (almost impossible for untextured objects).

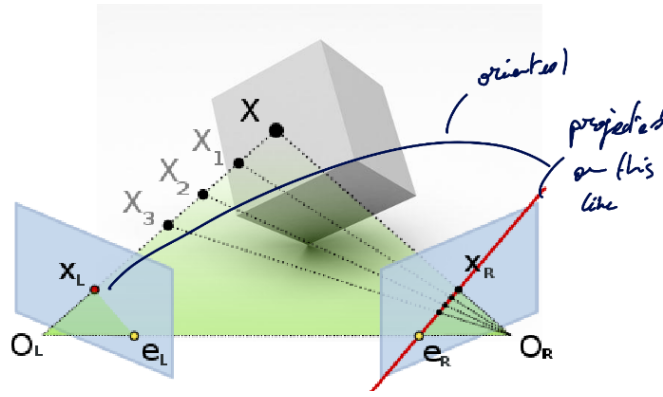


Figure 67: General setup

9.1.2.2. General Stereo vision Now, the planes are not necessarily parallel and the cameras are not necessarily on the same plane. We can use the epipolar geometry to find the correspondance between the two images. The idea is to find the epipolar lines in both images and then look for correspondance along these lines. This is called the **epipolar constraint**. The epipolar constraint states that the point in one image must lie on the epipolar line in the other image. We can then use the disparity along the epipolar line to find the depth of the scene. This method requires both camera to be properly calibrated and the relative position, orientation, and setting of the cameras to be known.

$$\mu p = KR^\top(P - C) \quad \rho' p' = KR'^\top(P - C') \quad \text{the Ray: } P = C + \mu RK^{-1}p \quad \mu \in \mathbb{R}$$

Not be studied but good for understanding. The point P must lie on the ray defined by the camera center and the point in the image. We can then find the epipolar line by finding the intersection of the ray with the image plane of the other camera using the equation $\rho' p' = KR'^\top(P - C')$. This is given by:

$$\rho' p' = KR'^\top(C + \mu RK^{-1}p - C') \quad \rho' p' = \underbrace{KR'^\top(C - C')}_{\text{Epipole}} + \underbrace{\mu KR'^\top RK^{-1}p}_{\text{Vanishing point}}$$

The epipole is the point where the line connecting the two camera centers intersects the image plane. The vanishing point is the point where the ray defined by the camera center and the point in the image intersects the image plane of the other camera. The epipolar line is then given by the line connecting the epipole and the vanishing point.

We can then use the simplified notation with A the infinite homography and e the epipole:

$$A = \frac{1}{\rho'_e} K' R'^\top R K^{-1} \quad e = \frac{1}{\rho'_e} K' R'^\top (C - C') \quad \rho' p' = \rho'_e (\mu A p + e')$$

We can write the epipolar constraint as:

$$|p' e' A p| = p'^\top (e' \times A p) = 0 \rightarrow |p' e' A p| = p'^\top [e']_\times A p = 0 \quad \underbrace{F = [e']_\times A}_{\text{Fundamental matrix}} \quad [e']_\times = \begin{bmatrix} 0 & -e'_z & e'_y \\ e'_z & 0 & -e'_x \\ -e'_y & e'_x & 0 \end{bmatrix}$$

The fundamental matrix F encapsulates the epipolar geometry of the two views. It is a 3x3 matrix which has rank 2. ($p'^\top F$) is a triple (a', b', c') giving the coefficients of the epipolar line for point p with coordinates $(x, y, 1)$: $a' \cdot x + b' \cdot y + c' = 0$. And vice versa for Fp .

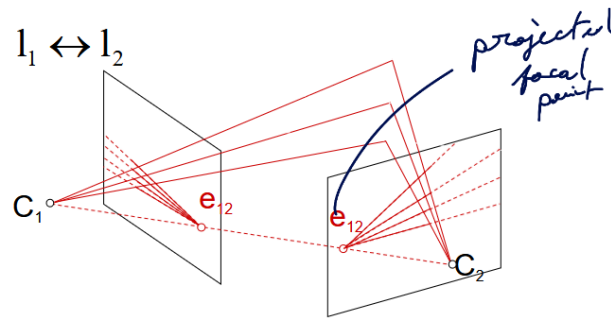


Figure 68: Epipolar lines are in mutual correspondence == allows to separate the matching problem

How to find the Fundamental matrix:

- One point yields one equation $p'^T F p = 0$, that is linear in the entries of the fundamental matrix F so, we can actually obtain F without any prior knowledge about camera settings if we have sufficient pairs of corresponding points !!
- F can be computed linearly from 8 pairs of corresponding points; not 9, as this is a homogeneous system of equations.
- F being rank 2 yields an additional, but non-linear constraint ($\det(F)=0$), thus 7 correspondences suffice to non-linearly solve for F

In practice,

- Detect interest points (using detector) that are good candidates to serve as initial correspondence and match features (e.g. SIFT/ORB/etc.)
- Solve the 8-point algorithm using e.g. least squares and find the point pairs that fit the model using RANSAC (this is to avoid false positives, RANSAC randomly selects initial pairs, and adds pairs gradually while the F -equation remains between bounds)
- Estimate final F (this might not have rank 2, because of noise, imperfect matches, measurement errors rank will almost always be full=3)
- Enforce rank 2 using SVD: decompose $F = U\Sigma VT$, singular values are in $\Sigma = \text{diag}(\sigma_1, \sigma_2, \sigma_3)$, and force $\sigma_3 = 0$.

Epipolar lines are in mutual correspondence. Separate 2D correspondence search problem to 1D search problem by using two view geometry

We can also reduce the search space by only picking points on the epipolar line, min and max depth for line segment, smoothness of the disparity field, ...

Again this is for reference and the math derivation presented here is not required for the exam, but it is good to understand the underlying principles of stereo vision and epipolar geometry. By re-using the ray equation and epipolar constraint we have 6 equations and 5 unknowns (over determined set of equations). Sometimes the ray may not intersect due to noise so we take the middle.

$$P = C + \mu R K^{-1} p \quad \mu \in \mathbb{R} \quad P = C' + \mu' R' K'^{-1} p' \quad \mu' \in \mathbb{R}$$

9.1.2.3. Multiview stereo We can expand this idea to multi-view stereo where we have a moving camera and we want to reconstruct the 3D scene from multiple images. We can use the same principles as stereo vision but we need to take

into account the motion of the camera. We can use structure from motion (SfM) to find the camera motion and then use multi-view stereo to reconstruct the 3D scene.

We must do feature tracking, estimate pairwise geometry, and then do multi-view stereo to reconstruct the 3D scene. This is a complex process but it allows us to reconstruct the 3D scene from multiple images. We must also self-calibrate to understand the intrinsic parameters of the camera K and the relative position and orientation of the cameras. Convert it to another base $E = K^\top FK$.

9.2. Active: interfere with the scene to get information

9.2.1. Uni-directional

9.2.1.1. Time-of-flight This is the idea behind radar or lidar, we measure the time-of-flight of a pulse or we analyze the phase shift of a continuous wave to find the distance of the object. It is a direct way to measure depth but it can be affected by noise, multipath interference, and other factors. We can cast multiple ray to get a point cloud of the scene. The distance is given by:

$$d = \frac{c \cdot t}{2} \quad d = \frac{c \cdot \phi}{4\pi f}$$

9.2.2. Multi-directional

9.2.2.1. Line scanning The idea is to simplify the stereo vision by projecting a line on the scene and then looking at the deformation of the line to infer the 3D structure of the scene. This is a simple but effective way to reconstruct the 3D scene. It is often used in industrial applications for quality control and inspection. We can cast a cloud of points and capture the deformation of the line to reconstruct the 3D scene. It is a direct way to measure depth but it can be affected by noise, occlusion, and other factors and find no intersection points !.

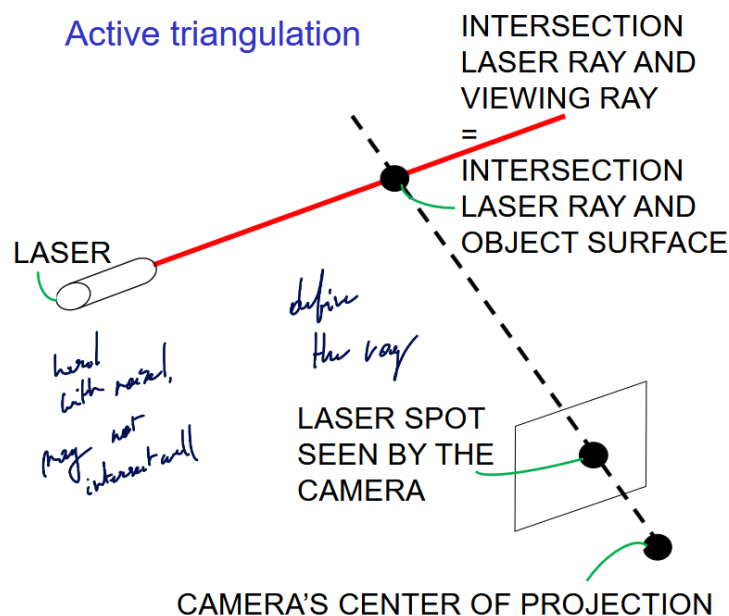


Figure 69: Active triangulation

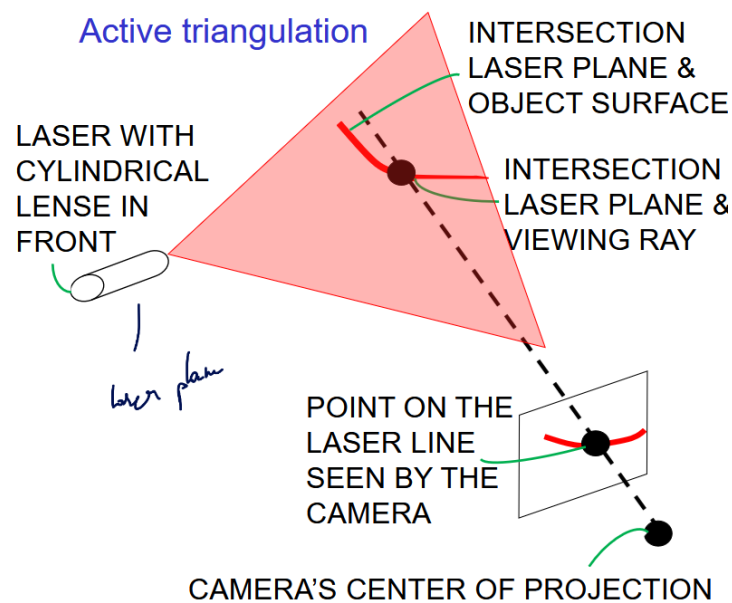


Figure 70: Active triangulation: with a plane, noise has less influence

9.2.2.2. Structured light The idea is to project a pattern of light on the scene and then look at the deformation of the pattern to infer the 3D structure of the scene. The goal is to have minimum number of pattern projection while keeping a good resolution. For example, we can use serial binary patterns with increasingly fine subdivisions 2^n . Or we can use colors 3^n . Finally we can use a checkerboard pattern with a column code giving one shot implementation. The kinect uses 9×9 with unique code.

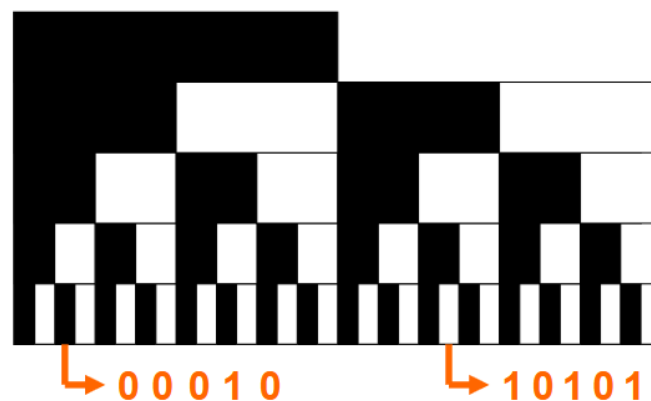


Figure 71: Serial binary patterns

9.2.2.3. Photometric stereo Again based on the lambertian assumption, we will change the light actively to observe the object under various lighting conditions. Can give really high contrast and depth for small objects with little texture.

Part II.

Modern Image Analysis

Table 7: Summary of notorious DeepLearning network

Type	Network of net-	Characteristics
name	work	
Alexnet CNN		8 layers, 60 million parameters, 1.2 million images, 1000 classes; handcrafted sizes
VGGNet CNN		19 layers, 138 million parameters, 1.2 million images, 1000 classes; use 3x3 CNN, when pooling double depth if we divide height and width by 2, more mature. Declined in model sizes VGG-XX with XX layers.
ResNet CNN		152 layers, 25 million parameters, 1.2 million images, 1000 classes; use residual connections to avoid vanishing gradient problem, more mature. Declined in model sizes ResNet-XX with XX layers. Use average pooling

Other normalization modules

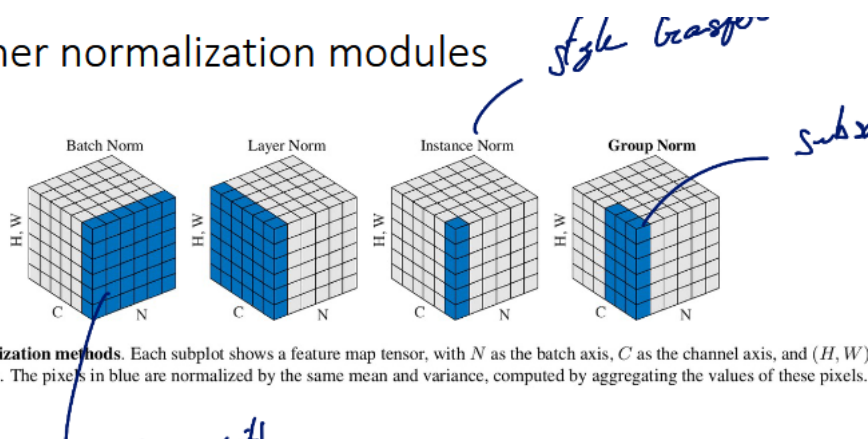


Figure 72: Type of normalization; N is the batch size, C are the channels, $H \times W$ are the spatial dimensions

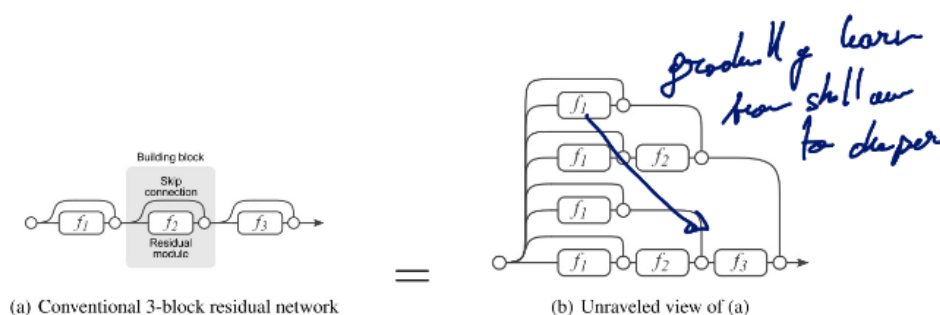


Figure 73: Resnet interpretation

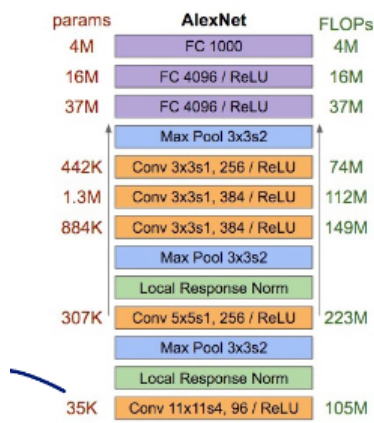


Figure 74: Size estimation for a CNN and its head

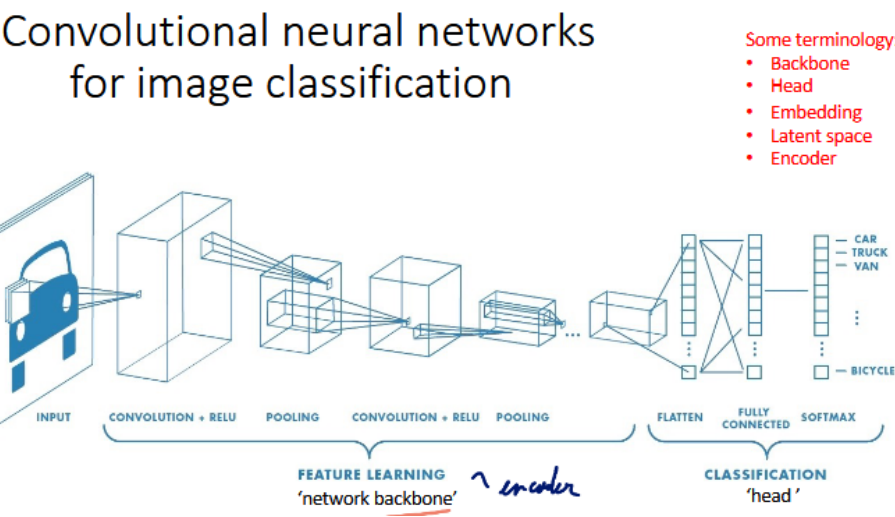


Figure 75: Typical CNN and its nomenclature

Table 8: Loss function and use case

Type of loss	Usecase	Math
softmax + Cross-entropy loss	Classification (1 class)	$L = - \sum_{i=1}^N y_i \log(\hat{y}_i)$
Sigmoid + Binary Cross-entropy loss	Classification (multi-label)	$L = - \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$
L1,L2,Huber loss	Regression (continuous value)	$L = \sum_{i=1}^N \ y_i - \hat{y}_i\ $ (L1), $L = \sum_{i=1}^N (y_i - \hat{y}_i)^2$ (L2, gradient can explode bad), $L = L2$ if $L1 \leq \delta$ else $L1$ (Huber)

Table 9: Object detection NN

Name		
of	Type	
NNet-	of	
work	model	Characteristics
R-CNN	2-stage	Region-based CNN, uses selective search to find region proposals and then applies a CNN to each proposal to classify it. Use SVM classification -> no end-to-end training. Simpler wrapper over AlexNet.
Fast R-CNN	2-stage	An improvement over R-CNN, it uses a single CNN to extract features from the entire image and then applies a region of interest (RoI) pooling layer to classify each proposal. Use a linear classifier (Cross-entropy for label classification and Huber Loss for regression (bounding box))
Faster R-CNN	2-stage	An improvement over Fast R-CNN, it uses a region proposal network (RPN) to generate region proposals instead of selective search, making it faster and more efficient. Use anchors; 2 NN 1 for classification 1 for region proposal based on fixed anchors (regression network)
Mask R-CNN	2-stage	An improvement over Faster R-CNN, it adds a branch for predicting segmentation masks on each region of interest, in addition to the existing branches for classification and bounding box regression.
YOLO	1-stage	You Only Look Once, it divides the image into a grid and predicts bounding boxes and class probabilities for each grid cell, making it faster but less accurate than 2-stage methods.
SSD	1-stage	Single Shot MultiBox Detector, it performs object detection in a single forward pass of the network, making it faster than 2-stage methods. Use anchor boxes that are learned. 300x300 inputs.
RetinaNet	1-stage	A single-stage detector that addresses the issue of class imbalance in object detection. It uses a focal loss function to focus on hard-to-classify examples.

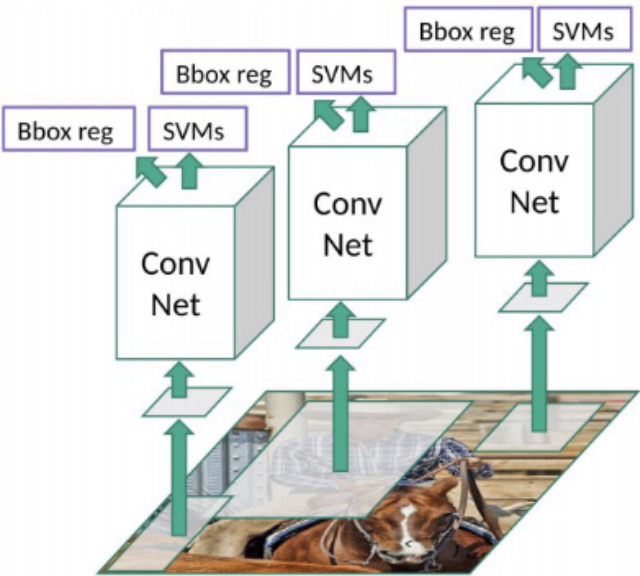


Figure 76: R-CNN

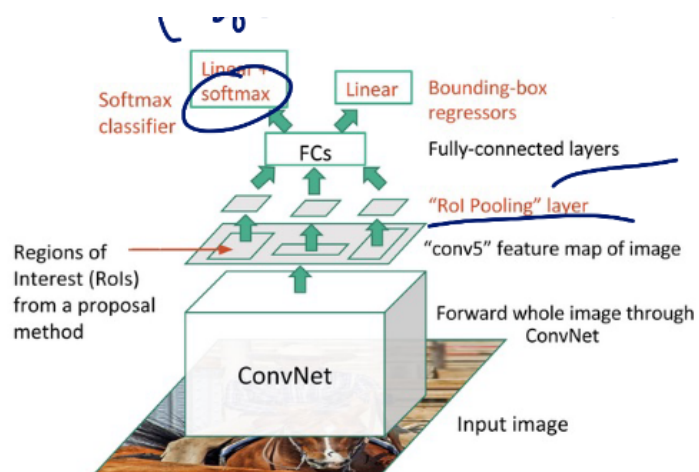


Figure 77: Fast R-CNN

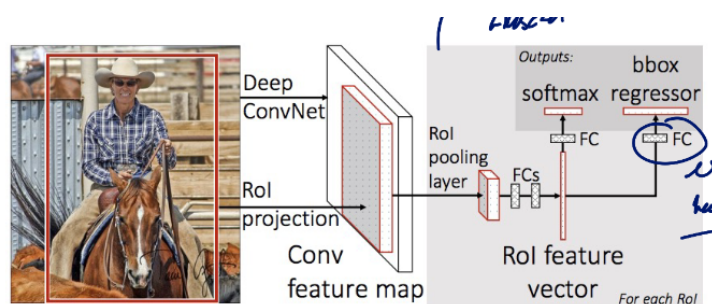


Figure 78: In-depth Fast R-CNN

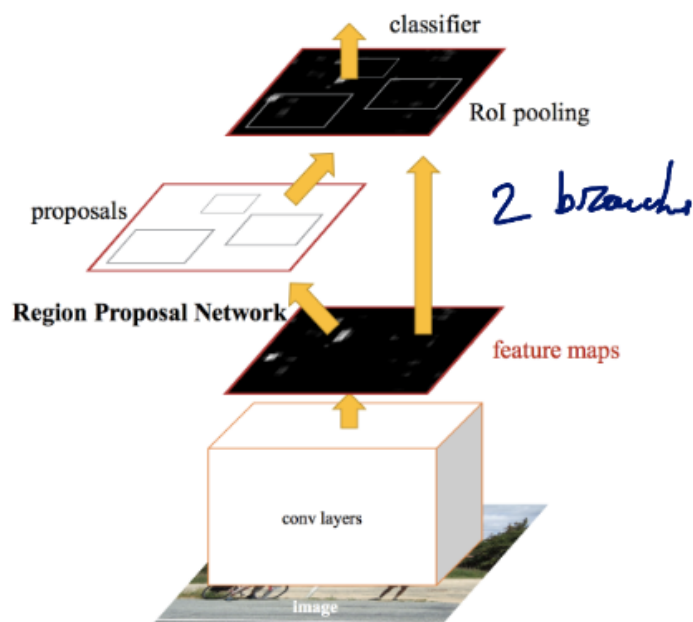


Figure 79: Faster R-CNN

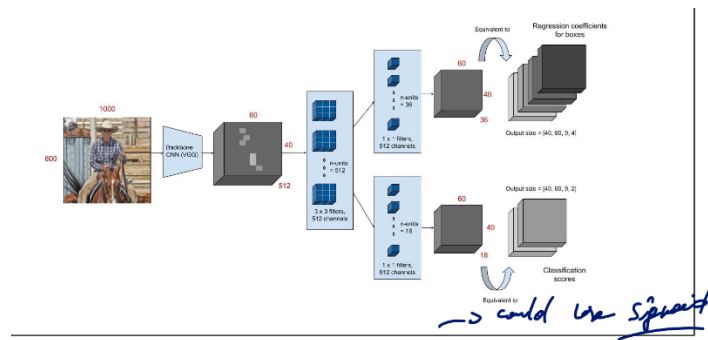


Figure 80: In-depth Faster R-CNN

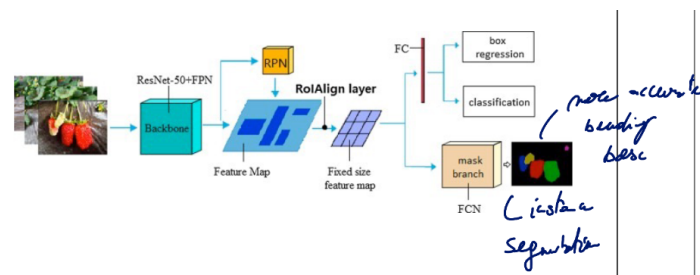


Figure 81: Mask R-CNN

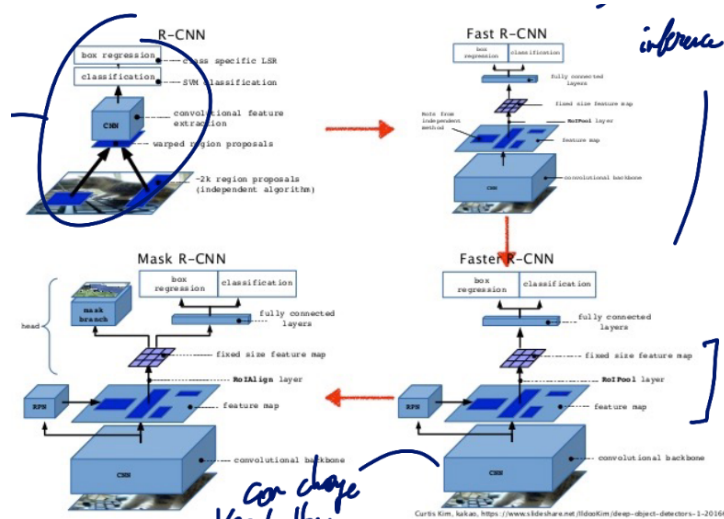
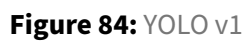


Figure 82: Summary of top-down detection

Model	Type	mAP (COCO-style)	FPS (GPU)	Latency	Strength
Faster R-CNN	Two-stage	80-95%	5-10 FPS	High	Highest accuracy
YOLO (modern)	One-stage	70-90%	30-100 FPS	Low	Best speed/accuracy trade-off
SSD	One-stage	60-80%	20-60 FPS	Medium	Simple + efficient

Figure 83: Comparison of Image detectors



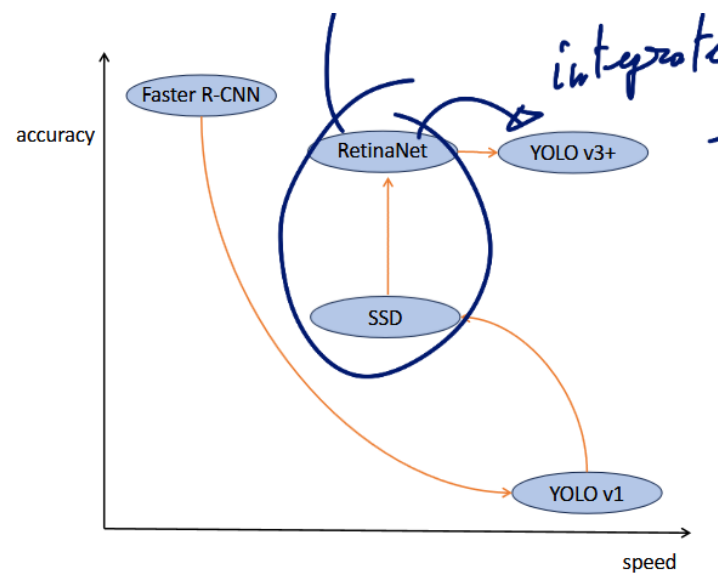


Figure 87: Overview of object detection

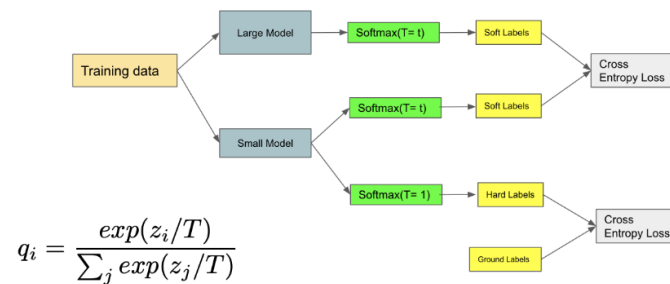


Figure 88: Knowledge distillation

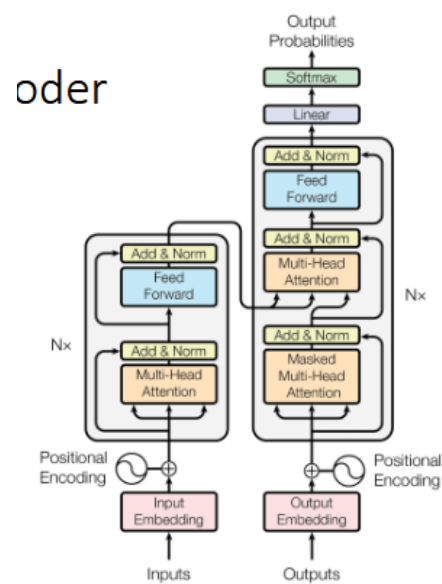


Figure 89: Encoder-Decoder; with the cross-attention layers

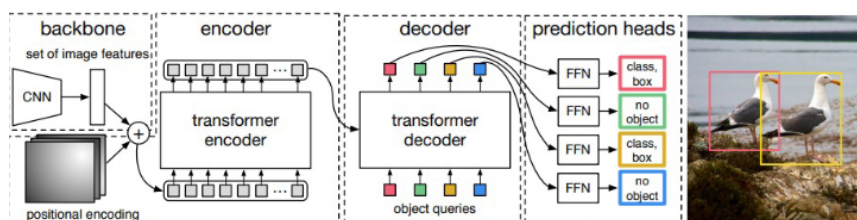


Figure 90: DETR

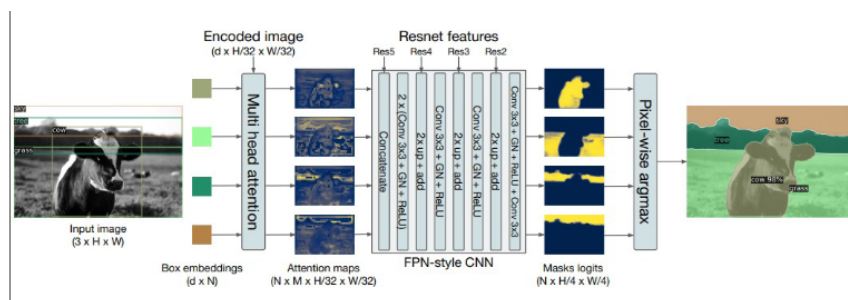


Figure 91: DETR + panoptic segmentation

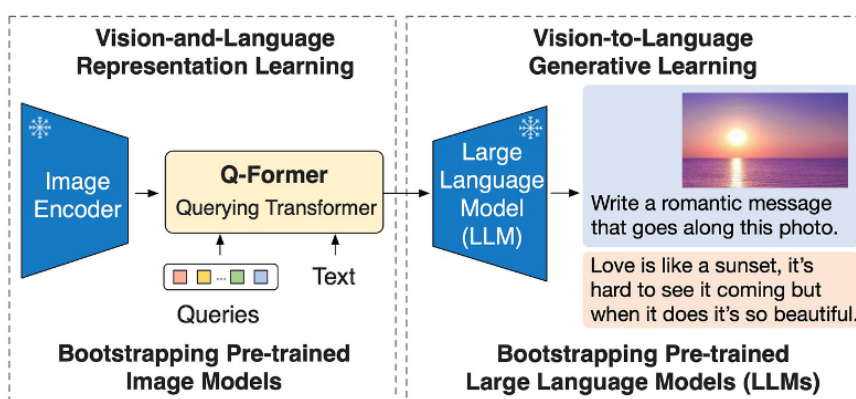


Figure 92: Transformers for multimodal tasks

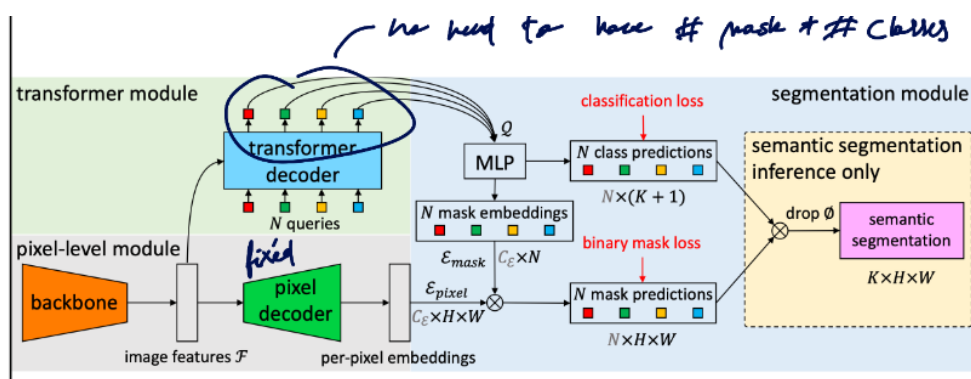
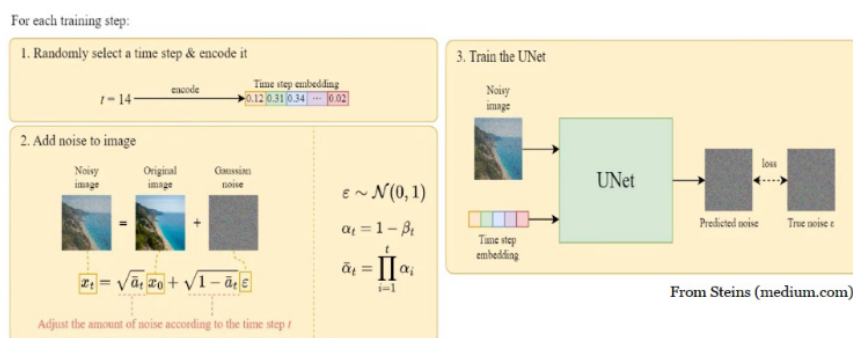


Figure 93: All together; vision transformer + decoder = MaskFormer

Table 10: Generative Models overview

Name	Type of Models	Characteristics
Autoencoder	AE	Learns a latent representation of the data and can generate new samples from this representation. Explicit model.
Variational Autoencoder	VAE	Like AE, but use the Kullback Leiber norm to regularize the latent space distribution to fit a gaussian distribution.
Generative Adversarial Network	GAN	Consists of a generator and a discriminator that compete against each other to generate realistic samples. Implicit, force input to be gaussian distribution, game-theory based training.
Conditional GAN	GAN	Like GAN, but allows for conditioning the generation process on additional information, such as class labels or text descriptions.
Cycle GANG	GAN	We use real input image, translated into another domain, and then back to the original domain. The other domain is used against the original domain to enforce cycle consistency. Allows to do style transfer.
Diffusion Models	DiffusionModels	the data generation process as a diffusion process, where noise is added to the data and then removed to generate new samples.
Stable Diffusion (latent diffusion)	Diffusion	Use an encoder-decoder architecture to generate images from text prompts. Sort of VAE + diffusion. We apply the diffusion procedure on the latent space.

**Figure 94:** Diffusion models: Training

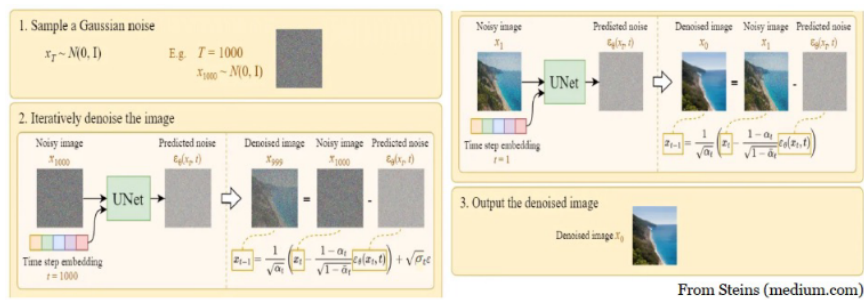


Figure 95: Diffusion models: Generation

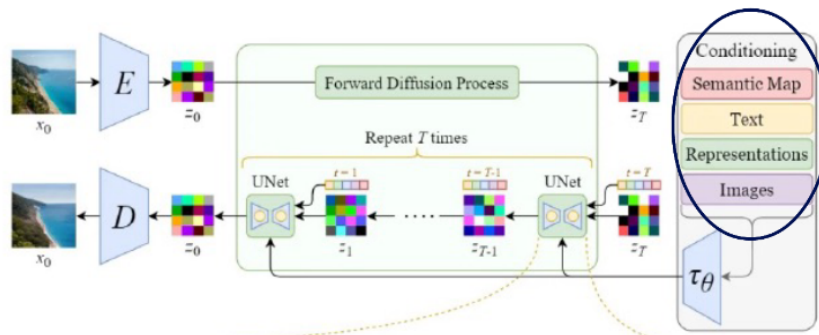


Figure 96: Latent diffusion

10. License

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0) for KU Leuven students.

You are free to:

- **Share** — copy and redistribute the material in any medium or format with people inside of KU Leuven or being granted access to the source material.
- **Adapt** — remix, transform, and build upon the material if you are a KU Leuven student.

Under the following terms:

- **Attribution** — You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- **NonCommercial** — You may not use the material for commercial purposes.
- **ShareAlike** — If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

For the full legal text of the license, please visit: <https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode>

10.1. Modification

To contribute to this work or any other one from this project please find more information at the Github repository.

10.2. Take down

If you are a professor or representing any official party and wish to take down this work, you can submit a takedown request at this url: https://github.com/Tfloow/ESATSummary/issues/new?template=takedown_notice.yml

Some Rights Reserved 2026 Authors of the Summary, Professors of the Course and possible book's authors. Some Rights Reserved.